

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П. О. СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

Специальность 6-05-0611-01 «Информационные системы
и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «Распределенные информационные системы»

на тему: **Семантический поиск в документации, связанной с организацией
учебного процесса**

Исполнитель: студент гр. ИТП-31
Куксёнок К.Е.

Руководитель: преподаватель
Комраков В.В.

Дата проверки: _____

Дата допуска к защите: _____

Оценка работы: _____

Подписи членов комиссии по защите
курсового проекта:

Гомель 2026

Установа адукацыі «Гомельскі дзяржаўны тэхнічны ўніверсітэт імя П.В. Сухого»

Факультэт аўтаматызаваных і інфармацыйных сістэм

Кафедра "Информационные технологии"

РЭЦЭНЗИЯ
на курсавы праект (работу)

па дысцыпліне «Разработка приложений баз данных»,

выканана студэнтам _____

групы _____

I. Пералік заўваг па тэксту курсавога праекта (работы)

II. Агульная характарыстыка работы

1. Своєчасовість виконання

+

-

нізкі ўзровень					выскі ўзровень				
1	2	3	4	5	6	7	8	9	10

нізкі ўзровень					высокі ўзровень				
1	2	3	4	5	6	7	8	9	10

нізкі ўзровень					высокі ўзровень				
1	2	3	4	5	6	7	8	9	10

нізкі ўзровень					высокі ўзровень				
1	2	3	4	5	6	7	8	9	10

нізкі ўзровень					высокі ўзровень				
1	2	3	4	5	6	7	8	9	10

нізкі ўзровень					высокі ўзровень				
1	2	3	4	5	6	7	8	9	10

(дата допуску, підпис адного з чальцоў камісії па праверцы)

нізкі ўзровень					высокі ўзровень				
1	2	3	4	5	6	7	8	9	10

« _____ » _____ 20____.
(дата)

О.Д. Асенчик

(подпісі, ініцыялы, П., І., І.п.б. чальцоў камісіі па праверцы)

СОДЕРЖАНИЕ

Введение.....	5
1 Обзор источников по семантическому поиску в документации, связанной с организацией учебного процесса.....	6
1.1 Обзор литературы и существующих методов решения задачи	6
1.2 Характеристика предметной области	7
1.3 Обзор архитектур систем семантического поиска и взаимодействие их компонентов	8
1.4 Программные средства реализации	10
2 Проектирование автоматизированной информационной системы	15
2.1 Проектирование информационной системы.....	15
2.2 Архитектура приложения.....	21
2.3 Логическая схема базы данных	24
2.4 Физическая схема базы данных.....	26
3 Программная реализация и тестирование информационной системы.....	28
3.1 Описание структуры и программных модулей приложения.....	28
3.2 Тестирование программной системы.....	35
3.3 Апробация разработанной системы	37
Заключение	38
Список использованных источников	39
Приложение А Листинг программы.....	40
Приложение Б Описание таблиц базы данных	50
Приложение В Внешний вид графического интерфейса программы	52

ВВЕДЕНИЕ

В современном мире непрерывно растущих объемов нормативной информации возникает острая необходимость создания эффективной системы семантического поиска в документах, регламентирующих поступление, пересдачи и отчисление в вузах Республики Беларусь. Предлагаемая разработка обеспечит быстрый доступ к релевантным правовым рекомендациям на естественном языке. Подобный инструмент имеет ключевое значение для строгого соблюдения законодательства и повышения качества управления учебным процессом.

Предложенный метод позволяет находить документы не по точному совпадению слов, а по смыслу запроса. Это особенно важно в критических ситуациях, например, когда речь идет о возможном отчислении или восстановлении в вузе. В такие моменты студенту не нужно знать точное название приказа или регламента – достаточно описать проблему своими словами. В связи с вышесказанным, эта задача актуальна.

Целью курсовой работы является создание программного продукта – распределенной системы, которая будет решать задачу семантического поиска по документации, связанной с организацией учебного процесса.

Для достижения поставленной цели необходимо решить следующий список задач:

- провести обзор источников по программно-техническому обеспечению, проанализировать предметную область и исследовать существующие подходы и методы решения задачи семантического поиска в текстовых документах;
- выполнить алгоритмический анализ задачи, то есть составить полную постановку задачи, определить перечень исходных данных и предполагаемых результатов, спроектировать обобщенную функциональную схему распределенного приложения, а также разработать иерархию классов и схему взаимодействия данных;
- осуществить программную реализацию разработанных алгоритмов, провести модульное и интеграционное тестирование созданных классов, а также выполнить верификацию и апробацию созданного приложения на реальных наборах данных.

1 ОБЗОР ИСТОЧНИКОВ ПО СЕМАНТИЧЕСКОМУ ПОИСКУ В ДОКУМЕНТАЦИИ, СВЯЗАННОЙ С ОРГАНИЗАЦИЕЙ УЧЕБНОГО ПРОЦЕССА

1.1 Обзор литературы и существующих методов решения задачи

В теории информационного поиска выделяются два фундаментальных подхода к извлечению релевантной информации: лексический и семантический поиск [1].

Классические системы лексического поиска, основанные на булевой логике и статистических метриках, осуществляют сопоставление поискового запроса с индексом документов по точному совпадению. Основным недостатком данного подхода является феномен «семантического разрыва» – неспособность системы распознать релевантность документа, если в нем используются синонимы или иные грамматические формы слов, отличные от введенных пользователем.

Для решения проблемы семантического разрыва в последние годы активно применяются методы обработки естественного языка (*Natural Language Processing*), базирующиеся на глубоких нейронных сетях и архитектуре *Transformer*. Нейросетевые модели преобразуют текст в плотные векторные представления (эмбединги), позволяя вычислять косинусное сходство между смысловым содержанием запроса и текста документа. Однако интеграция тяжеловесных языковых моделей предъявляет жесткие требования к аппаратно-программному обеспечению: необходимы значительные объемы оперативной памяти и вычислительные мощности графических процессоров (*GPU*). Данный фактор делает их применение нецелесообразным в рамках локальных систем с ограниченными ресурсами.

Альтернативным и высокоэффективным методом семантического поиска для узкоспециализированных предметных областей является применение систем, основанных на морфологическом разборе текста с последующим разбиением слов на векторы. В рамках данного подхода реализуются следующие методы:

- извлечение неизменяемой основы (корня) слова;
- связывание нормализованных корней исходного слова с корнями его синонимов на этапе индексации документа;
- оценка релевантности путем вычисления дистанции между словами (математическая оценка последовательностей слов).

В разрабатываемой системе принято решение отказаться от тяжеловесных нейросетевых решений в пользу разработки собственного программного морфологического анализатора, сопряженного с реляционной базой данных. Специфика обработки текстов на русском языке обусловлена его богатой словообразовательной природой. Стандартные алгоритмы извлечения корней (например, стеммер Портера), работающие по принципу усечения окончаний,

часто допускают ошибки переобучения и недообучения, что недопустимо при обработке строгой нормативно-правовой документации.

Для безошибочной идентификации лексической основы разработана модель анализатора, опирающаяся на словари морфем русского языка. Алгоритм выполняет структурный разбор форм слов в следующей последовательности:

- идентификация и отсечение окончаний, в которых выделяются морфемы, определяющие грамматические категории части речи, числа, рода и склонения;
- анализ словообразовательных суффиксов, в котором алгоритм рекурсивно отсекает суффиксы от полученной основы, учитывая их комбинации и способность влиять на принадлежность слова к определенной части речи;
- отсечение приставок, где производится поиск и обрезание исконных, заимствованных и составных префиксов;
- извлечение корня той части слова, которая осталась после вырезания приставки, окончания и суффикса.

1.2 Характеристика предметной области

Предметной областью разрабатываемой информационно-поисковой системы является нормативно-правовой текст, включающий в себя документации высших учебных заведений Республики Беларусь. Основу данного текста составляют положения, кодексы, уставы учреждений высшего образования, а также локальные правила. Указанные документы строго регламентируют порядок организации учебного процесса, проведения текущей и итоговой аттестации, перевода, восстановления и отчисления обучающихся.

С точки зрения автоматизированной обработки естественного языка и информационного поиска, рассматриваемая документация обладает рядом специфических характеристик, требующих особых подходов при проектировании поисковых алгоритмов.

Первой существенной особенностью является большой объем узконаправленной терминологии. Документы написаны в строгом официально-деловом стиле, предполагающем использование формализованных оборотов и сложных синтаксических конструкций. Описание образовательных и административных процессов осуществляется с помощью стандартных юридических формулировок, таких как «порядок ликвидации академической задолженности» или «досрочное прекращение образовательных отношений». Подобная специфика текстов практически исключает вероятность прямого лексического совпадения нормативных формулировок с обыденной речью целевой аудитории.

Из первой особенности закономерно вытекает проблема лингвистической неоднозначности запроса. Основными субъектами взаимодействия с проектируемой системой выступают студенты и абитуриенты. Данная категория пользователей формулирует поисковые запросы на неформальном, разговорном языке. В результате возникает смысловое и лексическое расхождение между текстом запроса и текстом нормативного акта. Данный фактор делает

применение классических систем полнотекстового поиска по точным ключевым словам крайне неэффективным и диктует необходимость внедрения специализированных словарей предметных синонимов.

Наконец, следует отметить сложную иерархическую структуру обрабатываемых документов. Локальные нормативные акты представляют собой объемные текстовые массивы, глубоко структурированные и разбитые на разделы, главы, статьи и пункты. Для обеспечения качественной выдачи системе недостаточно найти релевантный документ целиком, поскольку это заставит пользователя самостоятельно изучать многостраничный текст. Требуется точное извлечение конкретного абзаца, содержащего непосредственный ответ на введенный поисковый запрос.

Следовательно, проектируемая система должна выступать в роли семантического переводчика между разговорной лексикой пользователя и формальным языком нормативных актов. Оптимальным решением выявленных проблем является предварительное разбиение исходных текстов на смысловые абзацы с последующим морфологическим разбором каждого слова. Дополнительное автоматическое связывание извлеченных корней с заранее подготовленным электронным словарем предметных синонимов позволит с высокой точностью находить релевантные ответы, минимизируя время поиска правовых рекомендаций участниками образовательного процесса.

1.3 Обзор архитектур систем семантического поиска и взаимодействие их компонентов

Разработка автоматизированных систем информационного поиска требует тщательного проектирования архитектуры, определяющей принципы обработки, хранения и извлечения текстовых данных. Анализ современной научно-технической литературы позволяет выделить три основных архитектурных подхода к построению систем семантического и полнотекстового поиска: архитектуру на базе специализированных поисковых движков, микросервисную архитектуру с применением векторных баз данных и клиент-серверную архитектуру с многослойной организацией серверной части.

1.3.1 Архитектура на базе специализированных поисковых движков [2] предполагает использование готовых, высоконагруженных поисковых серверов (например, *Elasticsearch*, *Apache Solr* или *Sphinx*), выступающих в роли самостоятельного инфраструктурного узла. В основе таких систем лежит построение обратного индекса и применение вероятностных или статистических алгоритмов анализа текста.

Взаимодействие компонентов в рамках данной архитектуры осуществляется следующим образом:

- клиентское приложение (веб-интерфейс или настольный клиент) передает исходные текстовые документы в сервис-посредник;

- сервис-посредник осуществляет первичную очистку текста и направляет его по сетевому протоколу в специализированный поисковый движок;
- внутри поискового движка текст проходит через встроенный конвейер анализаторов;
- при выполнении запроса поисковый сервер самостоятельно вычисляет метрики релевантности и возвращает отсортированный список совпадений.

Преимуществом данной архитектуры является высокая скорость поиска по огромным массивам данных благодаря распределенному хранению индекса. Однако существенным недостатком выступает ресурсоемкость самого поискового сервера (требуются значительные объемы оперативной памяти для работы *JVM*) и сложность тонкой настройки лингвистических правил. Встроенные алгоритмы извлечения корня слова часто работают по принципу вероятностного отсечения окончаний, что затрудняет жесткую привязку предметных синонимов и точный морфологический разбор сложной юридической терминологии без написания кастомных плагинов.

1.3.2 Микросервисная архитектура с применением векторных баз данных [3] наиболее современный подход к реализации семантического поиска, который базируется на микросервисной архитектуре с использованием больших языковых моделей (*LLM*) и специализированных хранилищ векторных представлений. Данный подход позволяет системе улавливать смысл текста, игнорируя лексические несовпадения.

Взаимодействие компонентов в микросервисной архитектуре строится по следующему алгоритму:

- модуль управления документами извлекает текстовые абзацы и передает их в микросервис векторизации, внутри которого функционирует предварительно обученная нейронная сеть;
- нейронная сеть преобразует текст в многомерные массивы чисел с плавающей точкой (векторы признаков) и возвращает их в систему;
- полученные векторы вместе с метаданными документа сохраняются в векторной базе данных;
- при поступлении запроса от пользователя текстовая строка аналогичным образом конвертируется в вектор через микросервис векторизации;
- векторная база данных выполняет математическую операцию вычисления косинусного расстояния между вектором запроса и векторами всех сохраненных документов, возвращая наиболее близкие по смыслу результаты.

Данная архитектура обеспечивает наивысшую степень понимания естественного языка. Тем не менее, ее внедрение требует построения сложной инфраструктуры, обеспечения межсервисного сетевого взаимодействия и наличия специализированных аппаратных мощностей (графических

ускорителей). В контексте локальных или легковесных систем администрирования учебного процесса использование подобной архитектуры можно назвать избыточным.

1.3.3 Клиент-серверная архитектура [4] с многослойной организацией серверной части является альтернативой высоконагруженным поисковым кластерам и микросервисам. В рамках данного паттерна выделяются три базовых логических блока:

- клиентская часть, представляющая собой уровень демонстрации данных конечному пользователю (веб-браузер), который отправляет *HTTP*-запросы к серверу и отображает полученные *HTML*-страницы;
- серверная часть, реализуемая в виде веб-приложения, которое принимает запросы от клиента, выполняет тяжелую бизнес-логику и управляет взаимодействием с данными;
- сервер баз данных, обеспечивающий надежное хранение и целостность информации в классической реляционной СУБД.

Серверная часть в такой архитектуре проектируется на основе слоистого подхода. Приложение разбивается на независимые функциональные модули: модуль моделирования предметной области (модели), модуль обмена данными с базой данных (репозитории), модуль обработки данных (бизнес-логика), а также модули для обработки запросов и отправки ответов и представления пользовательского интерфейса (контроллеры и представления).

Таким образом, анализ существующих архитектурных решений демонстрирует, что для задачи обработки строгой нормативно-правовой документации в условиях ограниченных аппаратных ресурсов наиболее рациональным является использование клиент-серверной архитектуры с многослойной организацией серверной логики.

1.4 Программные средства реализации

Проектирование архитектуры системы семантического поиска требует обоснованного выбора стека технологий. Для обоснования выбора технологического стека целесообразно провести сравнительный анализ программных платформ, технологий маршрутизации и генерации пользовательских интерфейсов, систем управления базами данных, а также средств объектно-реляционного отображения и фреймворков для автоматизированного тестирования.

Для реализации алгоритмов обработки естественного языка и семантического поиска в современной индустрии преимущественно используются языки *Python* и *C#* (в рамках платформы *.NET*).

1.4.1 Язык *Python* [5] является стандартом в сфере машинного обучения и анализа данных. Его главным преимуществом является наличие обширной экосистемы библиотек для семантического анализа (*NLTK*, *spaCy*, *HuggingFace Transformers*), которые позволяют интегрировать готовые нейросетевые модели с минимальными трудностями. Динамическая типизация и простой синтаксис обеспечивают высокую скорость прототипирования. Однако интерпретируемая природа языка накладывает существенные ограничения на производительность при выполнении ресурсоемких математических вычислений и многопоточной обработке строк на стороне центрального процессора (*CPU*). Кроме того, динамическая типизация повышает риск возникновения ошибок времени выполнения при построении сложной бизнес-логики.

1.4.2 Платформа *.NET* и язык программирования *C#* [6] предлагают альтернативный подход, в котором *C#* является компилируемым, строго типизированным языком программирования. Ключевыми достоинствами платформы являются высокая производительность, эффективная работа сборщика мусора (*Garbage Collector*) и встроенная поддержка асинхронного и многопоточного программирования. Строгая типизация минимизирует количество ошибок на этапе компиляции, что критически важно при разработке сложных алгоритмов. Недостатком платформы *.NET* в контексте *NLP* является меньшее количество готовых решений для глубокого обучения по сравнению с *Python*.

Учитывая, что в рамках разрабатываемой системы принято решение отказаться от интеграции готовых нейронных сетей в пользу создания собственного морфологического анализатора, основанного на строгих правилах усечения частей слов и вычислении метрик релевантности, язык *C#* является предпочтительным.

Для реализации веб-ориентированной архитектуры «клиент-сервер» требуется фреймворк, обеспечивающий маршрутизацию запросов и генерацию *HTML*-представлений.

1.4.3 В экосистеме *Python* популярными решениями выступают фреймворки *Django* и *FastAPI* [7]. *Django* представляет собой монолитный фреймворк, реализующий паттерн *MTV (Model-Template-View)*. Он обладает богатым встроенным инструментарием, однако его монолитность приводит к избыточному потреблению ресурсов в системах, где требуется лишь обработка поисковых запросов. *FastAPI* является высокопроизводительным микрофреймворком, оптимальным для разработки *REST API*, однако он не содержит встроенных механизмов шаблонизации для генерации серверного *HTML*, требуя ручной интеграции сторонних библиотек.

1.4.4 В экосистеме *.NET* стандартом разработки веб-приложений является *ASP.NET Core MVC* [8]. Данный фреймворк реализует классический паттерн *Model-View-Controller*, обеспечивая строгое разделение ответственности между модулями обработки данных, бизнес-логики и представления. Существенным преимуществом *ASP.NET Core* является наличие мощного встроенного контейнера внедрения зависимостей (*Dependency Injection*), что позволяет легко управлять жизненным циклом сервисов морфологического анализатора. Использование механизма разметки *Razor* обеспечивает интеграцию *C#*-кода в *HTML*-документы для динамической генерации страниц с результатами поиска. Высокая производительность встроенного веб-сервера (*Kestrel*) делает *ASP.NET Core MVC* оптимальным выбором для построения серверной части разрабатываемой системы.

Обеспечение взаимодействия объектно-ориентированного кода (бизнес-логики) с реляционной базой данных требует применения *ORM*-фреймворков.

1.4.5 В экосистеме языка программирования *Python* стандартом для обеспечения объектно-реляционного отображения (*ORM*) выступает библиотека *SQLAlchemy* [9]. Данный инструмент предназначен для абстрагирования программного кода от специфики синтаксиса конкретных реляционных СУБД. Архитектурной основой фреймворка является реализация паттерна *Data Mapper*, который обеспечивает строгую изоляцию объектно-ориентированных моделей бизнес-логики от физической структуры хранения данных в таблицах. Библиотека обладает двухуровневой архитектурой, включающей базовый механизм генерации *SQL*-выражений (*SQLAlchemy Core*) и высокоуровневый *ORM*-компонент, что предоставляет разработчику исключительную гибкость при проектировании сложных схем баз данных и выполнении многоуровневых объединений (*JOIN*).

1.4.6 Для платформы *.NET* основным *ORM*-фреймворком является *Entity Framework Core* [10]. Инструмент базируется на технологии *LINQ*, которая позволяет формировать запросы к базе данных с использованием строго типизированного синтаксиса *C#*. Преимуществом *EF Core* является проверка корректности запросов на этапе компиляции приложения. Учитывая сложную структуру хранения морфем (выделение таблиц для корней, приставок, суффиксов и окончаний, связанных с уникальными словами), строгая типизация *LINQ*-запросов в *EF Core* существенно снижает риск нарушения целостности данных при их сохранении и извлечении.

Для хранения документов и выделенных морфем необходим выбор оптимальной СУБД. На этапе проектирования рассматривались классические

реляционные системы (*MySQL*, *Microsoft SQL Server*) и документно-ориентированные *NoSQL* решения (*MongoDB*).

1.4.7 СУБД *MongoDB* [11] хранит данные в формате *BSON* (бинарный *JSON*) и не требует жесткого задания схемы таблиц. Неоспоримым преимуществом *MongoDB* для задач современного информационного поиска является возможность интеграции с векторными алгоритмами: документы можно хранить в виде чанков вместе с многомерными эмбедингами слов, осуществляя математический поиск по векторам прямо на уровне БД. Однако документно-ориентированная модель не является оптимальной для хранения строго нормализованных реляционных связей между словами и их морфемным составом.

1.4.8 В качестве альтернативного варианта хранения структурированной информации широко применяется реляционная СУБД *MySQL* [12]. Данная система представляет собой классическое, высокопроизводительное серверное решение с открытым исходным кодом, базирующееся на строгой табличной архитектуре. Использование современных подсистем хранения (например, *InnoDB*) обеспечивает полноценную поддержку транзакционной модели *ACID* (атомарность, согласованность, изолированность, долговечность). В контексте семантического поиска это свойство гарантирует строгую ссылочную целостность при пакетном сохранении сложных иерархических структур, таких как взаимосвязи между уникальными словами документов и массивами выделенных из них морфем.

1.4.9 *Microsoft SQL Server* [13] является промышленным стандартом реляционных СУБД для платформы *.NET*. Важным достоинством данной системы является наличие *LocalDB*. *Microsoft SQL Server LocalDB* выполняется как фоновый процесс, который не требует сложного администрирования, настройки служб *Windows* и потребляет минимум оперативной памяти. Физическое хранение таблиц осуществляется в локальном файле *.mdf*. Поскольку разрабатываемая система опирается на нормализованную реляционную модель хранения данных (где слово связано с массивами корней, префиксов, суффиксов и окончаний посредством внешних ключей), использование *Microsoft SQL Server LocalDB* является наиболее рациональным выбором для развертывания локального программного комплекса.

Обеспечение отказоустойчивости морфологического анализатора и сервисного слоя требует внедрения инструментов автоматизированного тестирования.

1.4.10 В экосистеме языка *Python* индустриальным стандартом для обеспечения автоматизированного контроля качества программного кода, включающего модульное и интеграционное тестирование, выступает фреймворк *pytest* [14]. Отличительной особенностью данного инструмента является высокая декларативность и лаконичность синтаксиса, достигаемая за счет использования стандартного оператора *assert* без необходимости обязательного наследования тестовых классов от базовых компонентов фреймворка. Ключевым архитектурным преимуществом *pytest* признается мощная и масштабируемая система фикстур (*fixtures*). Данный механизм фактически реализует паттерн внедрения зависимостей (*Dependency Injection*) на уровне тестового окружения, позволяя эффективно и модульно осуществлять инициализацию, кэширование и последующую очистку (*teardown*) тестовых данных, а также безопасно изолировать работу с тестовыми хранилищами.

1.4.11 В экосистеме *.NET* для тестирования программных модулей используются фреймворки *NUnit*, *MSTest* и *xUnit* [15]. Современным стандартом является *xUnit*. Его архитектура построена на принципах максимальной изоляции тестов: фреймворк не использует общие методы предварительной настройки, а инициализирует новый экземпляр тестового класса для каждого тестового метода. Это исключает проблему взаимного влияния тестов друг на друга. Главным преимуществом *xUnit* в связке с *.NET* является нативная поддержка создания виртуальных баз данных в оперативной памяти средствами *Entity Framework Core*. Это позволяет проводить интеграционное тестирование слоя репозитория (*Data Access Layer*) с имитацией реальных *SQL*-транзакций, но без обращения к физическому жесткому диску, что обеспечивает скорость выполнения тестов на уровне миллисекунд. Данный инструментарий полностью покрывает потребности разрабатываемой системы в верификации бизнес-логики.

2 ПРОЕКТИРОВАНИЕ АВТОМАТИЗИРОВАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Проектирование информационной системы

Проектирование автоматизированной информационной системы семантического поиска начинается с детального анализа бизнес-процессов и определения функциональных требований. Для моделирования предметной области и описания логики взаимодействия компонентов системы была применена методология функционального моделирования *IDEF0*, а также унифицированный язык моделирования *UML*.

Диаграмма модели деятельности *TO-BE* с точки зрения ректора по методологии *IDEF0* на нулевом уровне показана на рисунке 2.1.

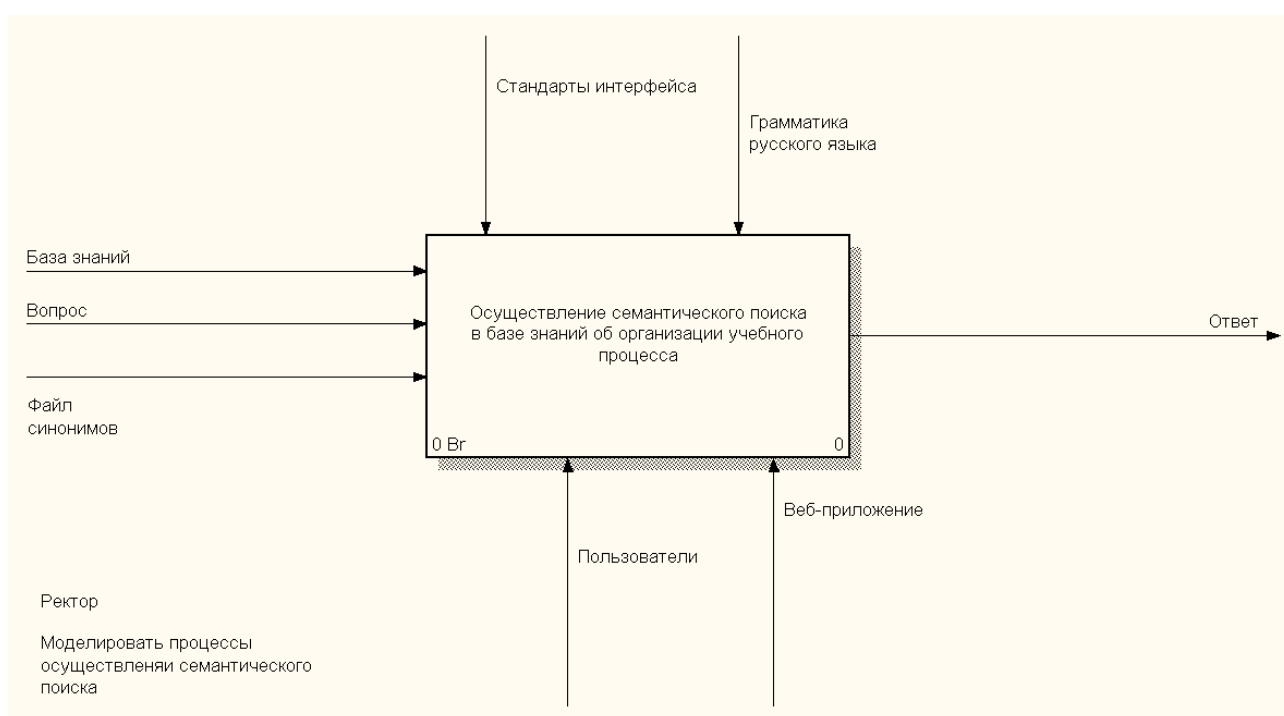


Рисунок 2.1 – Диаграмма модели деятельности по семантическому поиску на нулевом уровне

На вход системы поступает:

- база знаний, представляющая собой текстовые документы (нормативно-правовых актов, регламентов учебного процесса);
- файл синонимов, который содержит слова-синонимы по предметной терминологии;
- поисковый запрос, формируемый пользователем в виде текстовой строки на естественном языке.

Управление процессом семантического поиска осуществляется под строгим контролем грамматики русского языка и стандартов пользовательского веб-интерфейса. Под стандартами интерфейса подразумеваются алгоритмы

взаимодействия пользователя с браузером, а именно способы ввода текста в поля, механизмы выбора элементов из списков, вызов диалоговых окон для загрузки файлов и использование базовых элементов управления.

Всю деятельность семантического поиска осуществляет пользователь и веб-приложение, как автоматизированная система. На выходе системы формируется релевантный ответ (извлеченный из базы знаний абзац текста), который максимально точно соответствует смысловому контексту запроса.

Главной целью системы является моделирование и автоматизация процессов осуществления семантического поиска в локальной документации.

Для детализации внутренних процессов функционирования системы была разработана диаграмма декомпозиции первого уровня, представленная на рисунке 2.2.

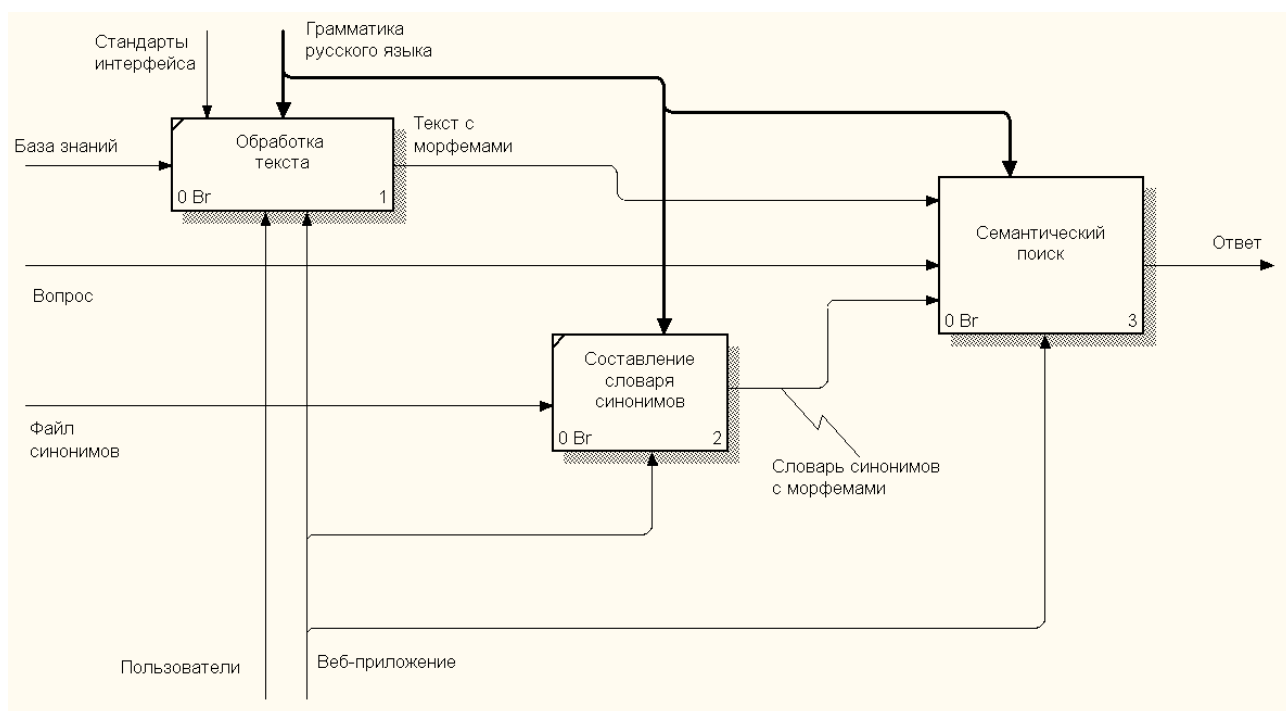


Рисунок 2.2 – Диаграмма декомпозиции на первом уровне

Весь процесс разбивается на три ключевых функциональных блока: «Обработка текста», «Составление словаря синонимов» и «Семантический поиск».

Блок «Обработка текста» предназначен для первичной обработки базы знаний. На вход данного блока поступают исходные документы под контролем правил грамматики русского языка и стандартов интерфейса. Выходом данного блока выступает форматированный текст, разбитый на морфемы.

Другим самостоятельным процессом является блок «Составление словаря синонимов». Его входом является файл синонимов, содержащий слова по теме организации учебного процесса. Под управлением грамматических правил система преобразует исходные текстовые пары в структурированный справочник. Выходом данного блока является словарь синонимов с

выделенными морфемами, где каждому корню сопоставлен массив родственных корней для последующего расширения поисковой выдачи.

Завершающим на данном уровне иерархии является блок «Семантический поиск». Его входами служат введенный пользователем поисковый запрос, а также результаты работы предыдущих блоков (текст с морфемами и словарь синонимов). Выполнение поиска строго контролируется правилами грамматики русского языка. Основным механизмом реализации для всех блоков выступает веб-приложение. Пользователь осуществляет свою деятельность только над блоком «Обработка текста» и вводом запроса. Результатом работы блока «Семантический поиск» является итоговый релевантный ответ.

Дальнейшая детализация подпроцесса «Семантический поиск» описана на диаграмме декомпозиции второго уровня, которая представлена на рисунке 2.3.

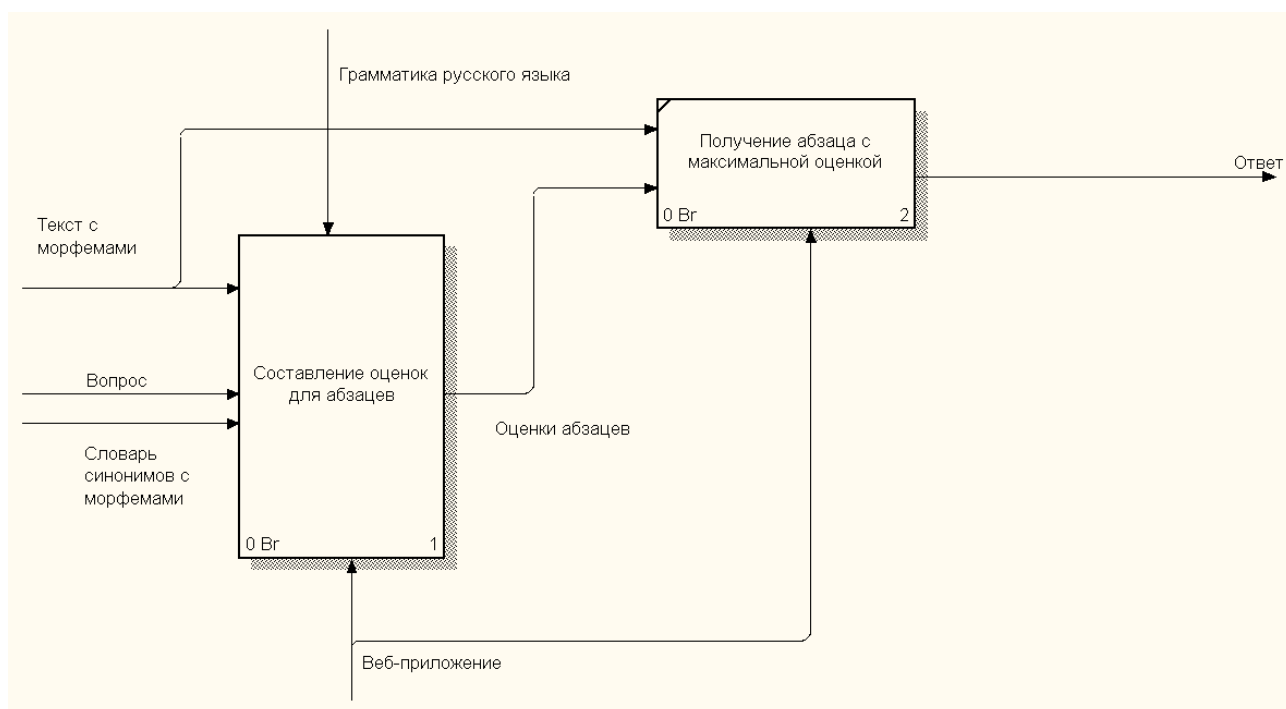


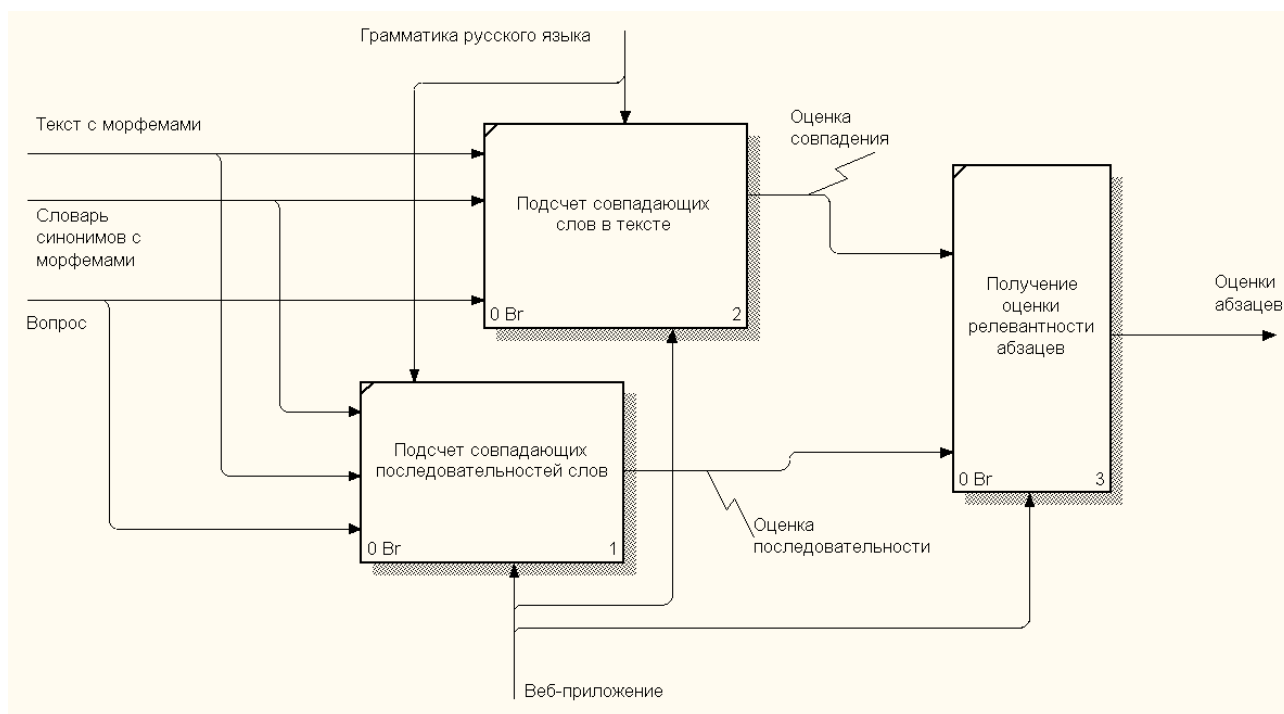
Рисунок 2.3 – Диаграмма декомпозиции активности «Семантический поиск» на втором уровне

Данный этап раскрывает внутреннюю архитектуру обработки пользовательского запроса и делится на два последовательных блока.

На вход блока «Составление оценок для абзацев» поступают поисковый запрос, сформированный словарь синонимов с морфемами и предварительно обработанный текст базы знаний (замена ё на е, приведения всего текста к нижнему регистру, обрезание «ненужных» слов и слов, содержащие меньше 3 букв). Контроль над выполнением процесса обеспечивается правилами грамматики русского языка. Выходом блока является массив числовых значений – расчетные оценки релевантности абзацев.

Сформированные на предыдущем шаге числовые оценки поступают на вход блока «Получение абзаца с максимальной оценкой». Алгоритм веб-

приложения осуществляет выборку текстового абзаца с наивысшим математическим показателем релевантности. Механизмом выполнения для обоих подпроцессов выступает серверная логика веб-приложения. Итоговым выходом системы формируется окончательный текстовый ответ, предоставляемый пользователю.



Вычислительный процесс разделен на три блока.

Наряду с оценкой одиночных совпадений работает блок «Подсчет совпадающих последовательностей слов». На основе вопроса, текста с морфемами и словаря с синонимами, алгоритм выполняет поиск слов из пользовательского запроса, идущих в тексте строго друг за другом. Выходом данного блока является оценка последовательности, которая существенно повышает итоговый балл релевантности абзаца при точном совпадении искомой фразы.

Блок «Получение оценки релевантности абзацев» выступает в роли агрегатора. Он принимает на вход такие промежуточные метрики, как оценка совпадения и оценка последовательности. Внутри блока происходит их математическое суммирование. Механизмом реализации для всех вычислительных блоков данного уровня выступает веб-приложение. Выходом является финальный набор числовых оценок абзацев, передаваемый на более высокий уровень иерархии системы для окончательной выборки.

Для формализации функциональных требований, описания сценариев поведения и определения границ проектируемой информационной системы разработаны диаграммы вариантов использования (*Use Case Diagram*) унифицированного языка моделирования *UML*. В рамках ролевой модели разрабатываемого программного комплекса выделены два независимых актора, инициирующих базовые бизнес-процессы: «Администратор» и «Пользователь».

Актор «Администратор» выполняет служебные функции по первичному наполнению информационной базы системы. Диаграмма вариантов использования администратора представлена на рисунке 2.5.

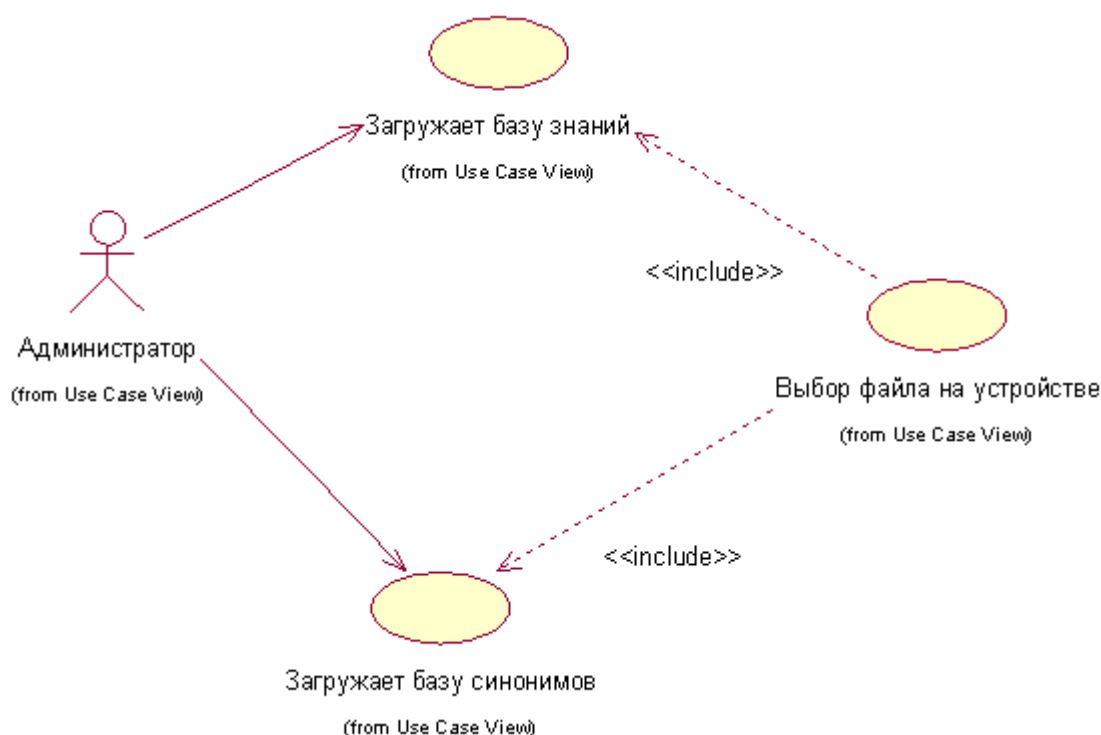


Рисунок 2.5 – Диаграмма вариантов использования администратора

Первым базовым прецедентом является «Загружает базу знаний», который отвечает за добавление новых текстовых документов в систему. Данный вариант использования связан отношением строгого включения (*<<include>>*) с прецедентом «Выбор файла на устройстве». Использование паттерна включения означает, что процесс загрузки базы знаний не может быть инициирован без обязательного обращения к локальной файловой системе пользователя через

стандартные элементы управления веб-браузера. На физическом уровне выполнение данного прецедента запускает внутренние алгоритмы сервера по чтению форматов (.txt, .docx, .pdf), очистке сырого текста и его передаче в модуль морфологического разбора.

Вторым прецедентом администратора является функция «Загружает базу синонимов». Наличие данного сценария позволяет системе обновить базу знаний предметной терминологии. В ходе выполнения прецедента система считывает справочник синонимов, выполняет морфологический разбор каждого синонима и устанавливает реляционные связи между однокоренными словами в базе данных.

Актор «Пользователь» является основным потребителем целевого функционала и взаимодействует исключительно с публичной частью системы. Диаграмма вариантов использования пользователя представлена на рисунке 2.6.

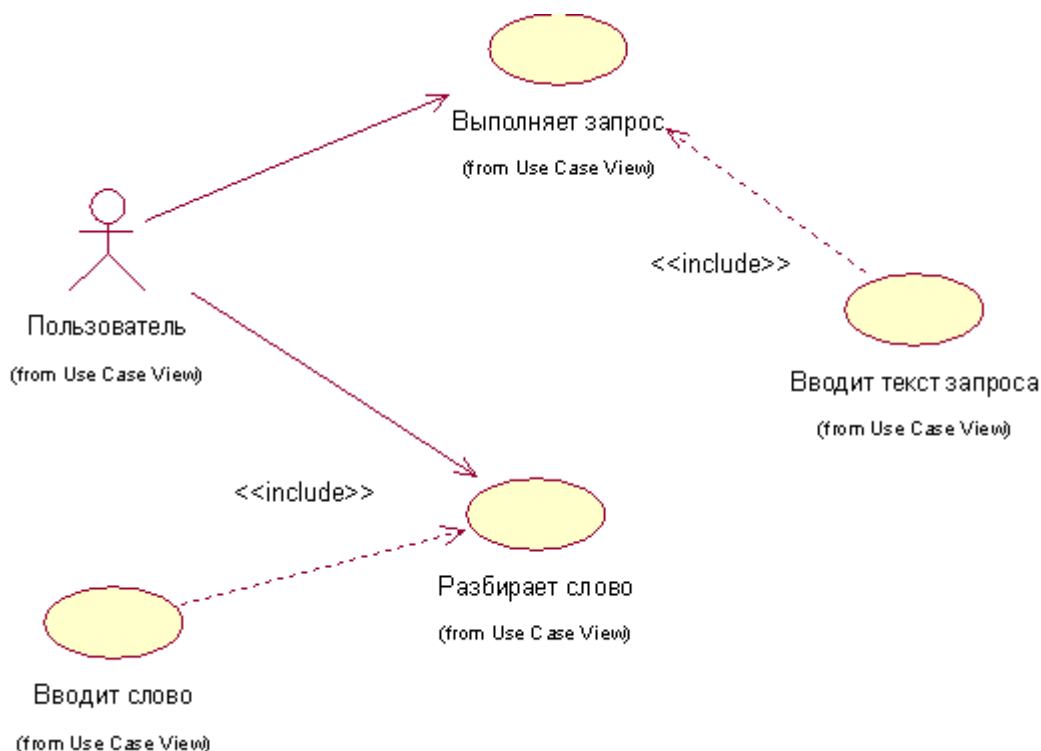


Рисунок 2.6 – Диаграмма вариантов использования пользователя

Для данного актора определен прецедент «Выполняет запрос», целью которого является извлечение релевантного абзаца.

Выполнение прецедента «Выполняет запрос» декомпозируется посредством связи <<include>>. Функция «Вводит текст запроса» осуществляет непосредственное взаимодействие пользователя с графическим интерфейсом, заключающееся в наборе текстовой строки и отправке данных на обработку.

Пользователь также может задействовать прецедент «Разбирает слово». Его выполнение невозможно без функции «Вводит слово», поскольку для

морфологического разбора нужно хотя бы ввести слово. Введенная лексема нормализуется, и разбивается на корни, приставки, суффиксы и окончания.

Для демонстрации распределения ролей и обмена данными в системе разработана обобщенная диаграмма взаимодействия, которая представлена на рисунке 2.7.

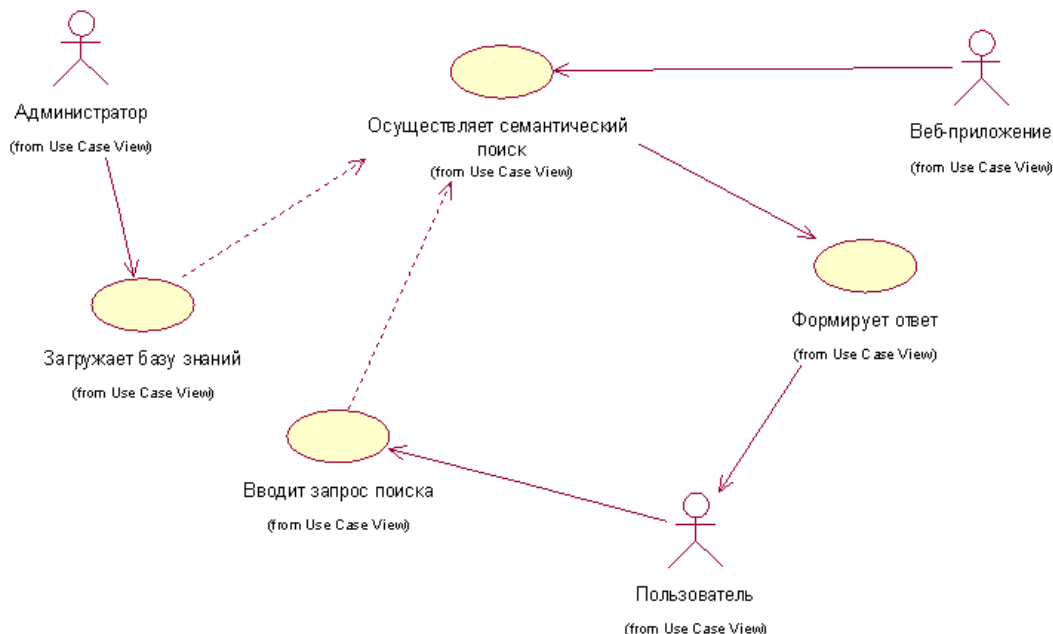


Рисунок 2.7 – Диаграмма взаимодействия

На данной диаграмме зафиксировано взаимодействие обоих акторов с центральным системным компонентом – программным узлом «Веб-приложение».

Актор «Администратор» инициирует процесс «Загружает базу знаний». После наполнения системных справочников актор «Пользователь» запускает функцию «Вводит запрос поиска». Центральный узел принимает данный прецедент и активирует главную системную функцию «Осуществляет семантический поиск». На данном этапе вычисляются числовые метрики совпадения слов и каждый абзац текстового документа получает свой числовой критерий. Завершающим этапом взаимодействия является активация узлом «Веб-приложение» прецедента «Формирует ответ», в результате которого пользователь получает готовую *HTML*-страницу с извлеченным фрагментом оригинального документа.

2.2 Архитектура приложения

В основе функционирования информационно-поисковой системы лежит классическая архитектура «клиент-сервер». Она объединяет удобный

пользовательский веб-интерфейс на стороне клиента и серверную часть, отвечающую за маршрутизацию, хранение данных и обработку поисковых запросов. Структурно архитектуру приложения можно разделить на три основных блока.

Клиентская часть представляет собой уровень визуализации данных конечному пользователю. В качестве клиента выступает любой современный веб-браузер, который отправляет *HTTP*-запросы к серверу, отображает сгенерированные *HTML*-страницы, а также реализует логику асинхронного обновления интерфейса с использованием технологии *Fetch API*.

Серверная часть реализована в виде веб-приложения на платформе *ASP.NET Core MVC*. Данный узел принимает запросы от клиента, выполняет основную бизнес-логику (включая алгоритмы морфологического анализа) и управляет взаимодействием с данными.

Сервер баз данных обеспечивает структуризацию, надежное хранение и ссылочную целостность извлеченных морфем, а также оригинальных текстов нормативных документов. В качестве системы управления базами данных (СУБД) используется *Microsoft SQL Server LocalDB*.

Общая схема архитектуры программного комплекса представлена на рисунке 2.8.

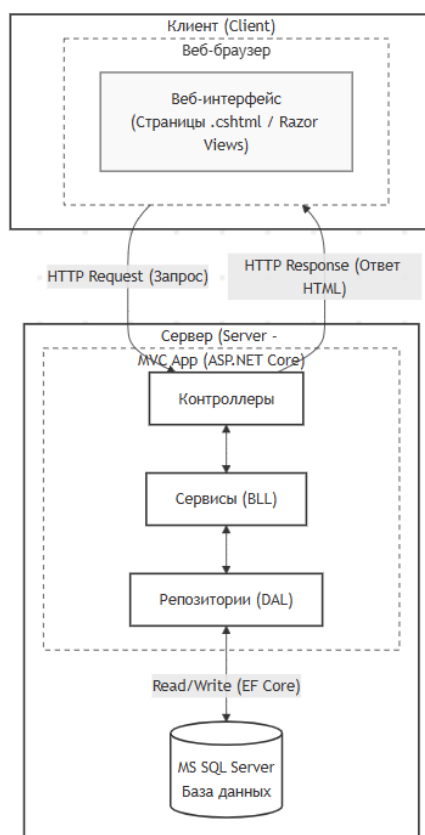


Рисунок 2.8 – Схема архитектуры программного комплекса

В рамках проекта использован фреймворк *ASP.NET Core* с языком программирования *C#*. Веб-интерфейс представлен в виде *CSHTML*-страниц, которые обрабатываются на сервере механизмом разметки *Razor* для генерации итогового *HTML*-кода. Хранение и обработка информации реализованы с использованием СУБД *Microsoft SQL Server LocalDB*. Взаимодействие между объектно-ориентированным кодом приложения и реляционной базой данных осуществляется посредством *ORM*-фреймворка *Entity Framework Core*.

Внутренняя структура серверного приложения спроектирована с использованием многослойной архитектуры. Суть данного подхода заключается в том, что весь программный код логически разделяется на независимые слои (модули), каждый из которых выполняет свою определенную задачу. Всего в системе выделено пять функциональных модулей:

- модуль моделирования предметной области (*Models*);
- модуль обмена данными с базой данных (*Repositories*);
- модуль бизнес-логики (*Services*);
- модуль маршрутизации и обработки *HTTP*-запросов (*Controllers*);
- модуль представления пользовательского интерфейса (*Views*).

Логическая структура взаимодействия программных модулей представлена на рисунке 2.9.



Рисунок 2.9 – Логическая структура приложения

Модуль моделирования предметной области содержит классы, которые являются точной проекцией таблиц реляционной базы данных в объектно-ориентированную среду языка *C#*. Данный слой не содержит бизнес-логики и используется исключительно для типизации данных.

Модуль обмена данными с базой данных (репозитории) отвечает за абстрагирование логики доступа к СУБД. Классы данного модуля реализуют обобщенный интерфейс *IRepository<T>*, инкапсулируя в себе методы генерации *LINQ-to-Entities* запросов, отслеживание изменений и выполнение *CRUD*-операций.

Модуль бизнес-логики представлен сервисным слоем (*Services*), который выступает связующим звеном между контроллерами и репозиториями, а также морфологическим анализатором (*Analyzer*). Анализатор инкапсулирует алгоритмы усечения приставок, суффиксов и окончаний, формирует вектор слова и вычисляет метрики релевантности.

Модуль обработки запросов (контроллеры) принимает входящие *GET* и *POST* запросы от клиента, осуществляет первичную валидацию данных (включая проверку целостности загружаемых файлов форматов *.txt*, *.docx*, *.pdf*), делегирует выполнение задач сервисному слою и выбирает способ отображения результата (возврат *HTML*-представления или *JSON*-ответа).

Модуль представления пользовательского интерфейса отвечает за генерацию финальных веб-страниц на основе полученных моделей данных, которые отправляются в браузер клиента.

2.3 Логическая схема базы данных

Для быстрого извлечения морфем слов в процессе поиска была спроектирована реляционная структура базы данных. Логическая модель опирается на принцип выделения каждой морфемы слова в отдельную сущность.

В рамках системы выделены следующие основные информационные сущности:

Документы – сущность для хранения метаданных загруженных файлов и их оригинального текстового содержимого. Содержит атрибуты идентификатора, названия документа, даты добавления и полного текста.

Все последовательные слова из текста – сущность, которая фиксирует каждое слово документа с сохранением порядка следования. Хранит само слово и ссылку на документ-источник.

Слова не дубликаты – центральная сущность морфологического словаря. Хранит уникальные нормализованные слова в нижнем регистре, выступая связующим звеном для всех выделенных морфем.

Корни – сущность, хранящая неизменяемую лексическую основу слова, полученную в результате работы анализатора, а также идентификатор части речи.

Приставки – сущность для хранения префиксов, отсеченных от оригинального слова.

Суффиксы – сущность, описывающая словообразовательные суффиксы. Содержит такие атрибуты, как суффикс, тип суффикса, наличие окончания и часть речи.

Окончания – сущность для хранения окончаний слова, включая грамматические атрибуты рода, числа и склонения.

В таблице 2.1 представлены все сущности логической модели и ее атрибуты.

Таблица 2.1 – Сущности и ее атрибуты

Сущности	Атрибуты
Документы	Идентификатор, название, дата добавления, оригинал текста
Все последовательные слова из текста	Идентификатор, слово, документ
Слова не дубликаты	Идентификатор, оригинал слова
Корни	Идентификатор, корень, часть речи, слово
Приставки	Идентификатор, приставка, часть речи, слово
Суффикс	Идентификатор, суффикс, тип суффикса, суффикс с окончанием, часть речи, слово
Окончание	Идентификатор, окончание, род, число, склонение, часть речи, слово

На основе выделенных сущностей была построена схема отношений, определяющая правила взаимодействия объектов внутри системы.

Связь «Документы» – «Все последовательные слова из текста» имеет тип «один-ко-многим». Один текстовый документ состоит из сотен или тысяч слов, порядок которых критически важен для алгоритма вычисления последовательностей. При этом каждое конкретное последовательное слово в этой таблице неразрывно привязано строго к одному документу.

Связи «Слова не дубликаты» – «Корни», «Слова не дубликаты» – «Приставки», «Слова не дубликаты» – «Суффиксы» и «Слова не дубликаты» – «Окончания» реализованы по типу «один-ко-многим». Одно и то же уникальное слово может быть разделено на несколько приставок или суффиксов. Кроме того, на этапе расширения запроса к уникальному слову искусственно привязываются корни его синонимов из предметного словаря. Таким образом, одному слову в таблице уникальных слов соответствует множество записей в таблицах морфем, но каждая конкретная морфема в базе всегда ссылается на оригинальное слово.

Схема логической модели базы данных, отражающая описанные выше сущности, их атрибуты и связи между ними, представлена на рисунке 2.10.

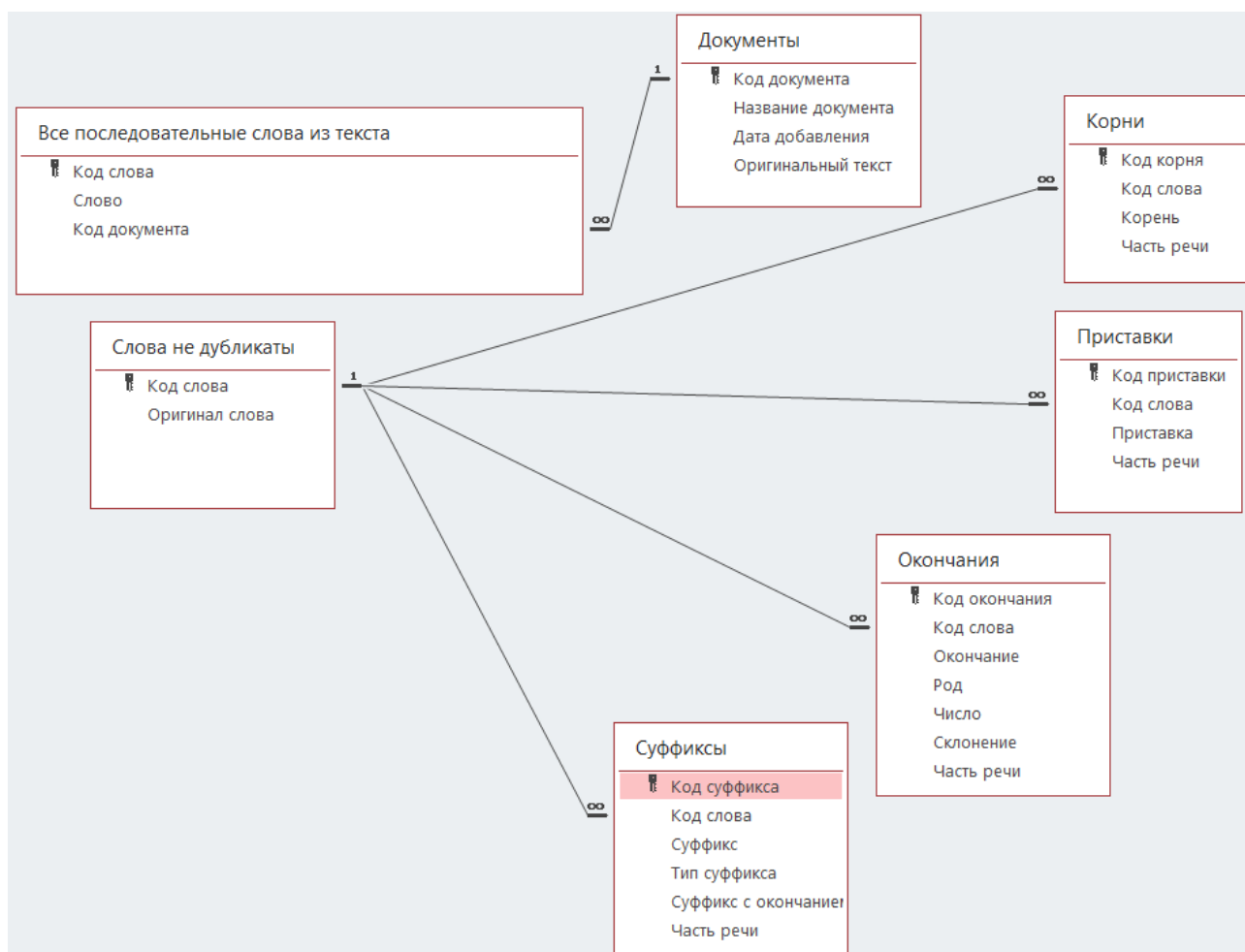


Рисунок 2.10 – Логическая схема базы данных

2.4 Физическая схема базы данных

На основе разработанной логической модели была спроектирована физическая структура базы данных для СУБД *Microsoft SQL Server LocalDB*.

В процессе преобразования логической модели в физическую структуру была обеспечена нормализация схемы до уровня третьей нормальной формы (3NF), что исключает избыточность хранения данных. Физическая реализация базы данных осуществляется средствами СУБД *Microsoft SQL Server LocalDB* и размещается на жестком диске в виде двух файлов: основного хранилища данных (*.mdf*) и журнала транзакций (*.ldf*).

При переводе логических атрибутов в физические поля таблиц применялись специфические типы данных *T-SQL*, обеспечивающие оптимальное хранение текстовой информации и числовых идентификаторов:

- целочисленные данные (*int*) используются для всех первичных (*Id*) и внешних ключей, а также для хранения данных перечислений (*Enum*), таких как *PartOfSpeechId*, *DeclensionId*, *GenderId*;
- строковые данные (*nvarchar(MAX)*) используются для хранения текстов документов, оригинальных слов и текстовых представлений морфем;
- дата и время (*DATETIME2*) применяются для фиксации временных меток добавления документов в систему (*DateAdded*).

Для обеспечения целостности и валидности данных физическая структура включает в себя ряд ограничений:

- *PRIMARY KEY* – каждая спроектированная таблица имеет кластерный индекс по первичному ключу, что организует физическое упорядочивание строк на диске и ускоряет поиск по идентификатору (метод *Find*);
- *FOREIGN KEY* – связи между таблицами жестко закреплены ограничениями внешних ключей, что гарантирует ссылочную целостность;
- *NOT NULL* – для обязательных полей установлено ограничение, запрещающее запись пустых значений (*NULL*).

Визуальное представление физической схемы базы данных, отражающее структуру таблиц, поля, типы данных и связи (*Primary/Foreign Keys*), приведено на рисунке 2.11.

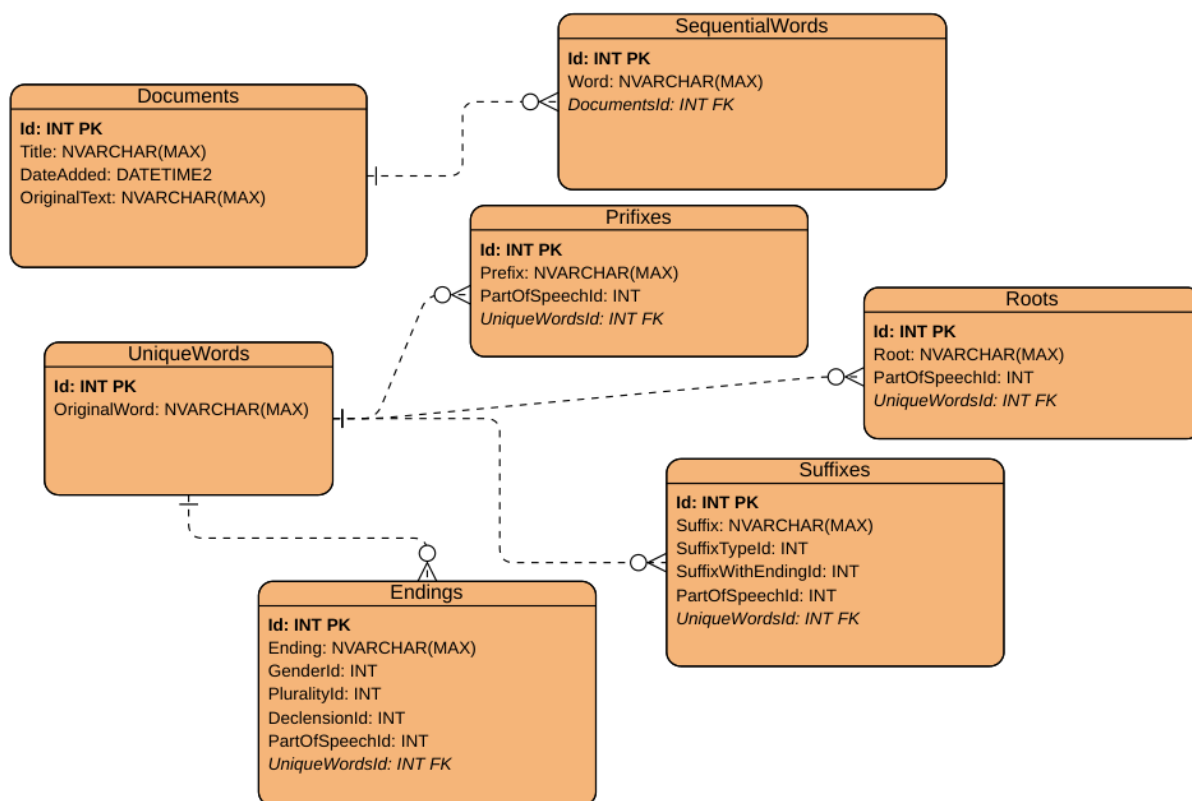


Рисунок 2.11 – Физическая схема базы данных

Полное описание состава полей всех разработанных таблиц, включая типы данных, их размеры и спецификации ограничений, вынесено в приложение Б.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Описание структуры и программных модулей приложения

Программная реализация системы семантического поиска выполнена на платформе *.NET 8* с использованием языка программирования *C#* и паттерна проектирования *Model-View-Controller*. Для обеспечения масштабируемости, строгой инкапсуляции бизнес-логики и удобства тестирования исходный код разделен на изолированные функциональные модули.

Фундаментом приложения выступает модуль объектно-реляционного моделирования (*Models*). Он предназначен для описания схемы базы данных средствами языка *C#* с применением технологии *Entity Framework Core*. Диаграмма классов моделей представлена на рисунке 3.1.

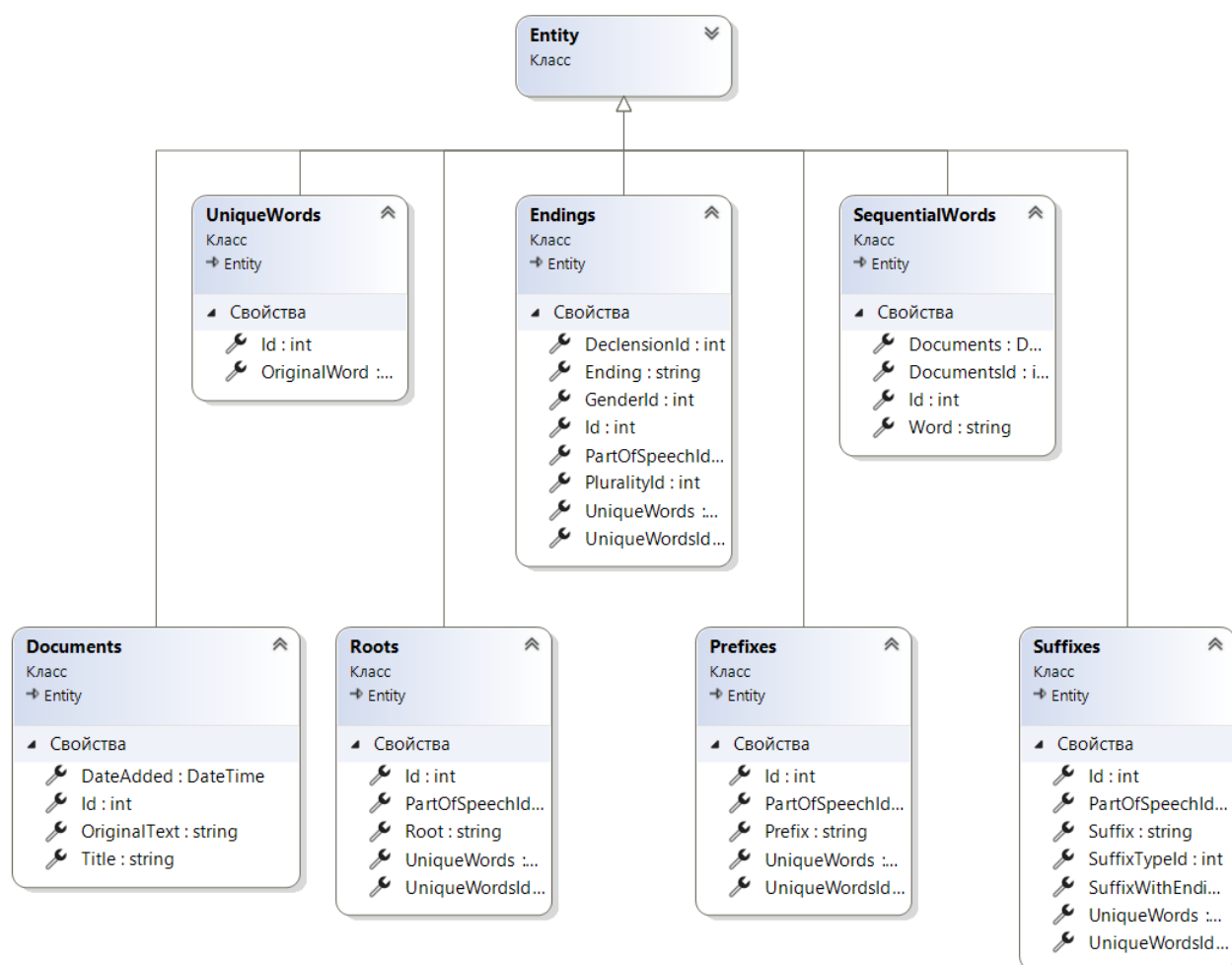


Рисунок 3.1 – Диаграмма классов моделей

В основе структуры лежит базовый класс *Entity*, от которого наследуются все классы предметной области, а именно *Documents*, *SequentialWords*, *UniqueWords*, *Roots*, *Prefixes*, *Suffixes*, *Endings*. Данные классы содержат свойства, определяющие структуру полей таблиц, и навигационные свойства для

настройки связей типа «один-ко-многим». Использование строго типизированных моделей гарантирует корректность форматов и защищает от структурных ошибок при сохранении извлеченных семантических данных в СУБД.

Для цифрового представления грамматических и лингвистических правил русского языка разработан модуль перечислений (*Enums*). Диаграмма перечислений представлена на рисунке 3.2.

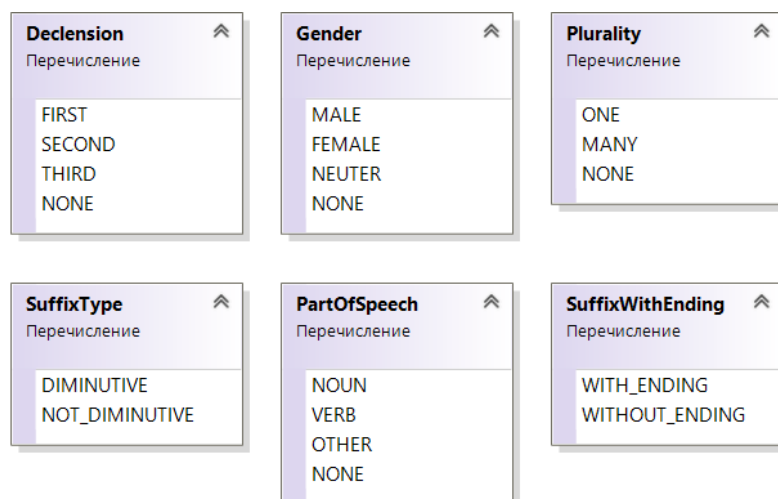


Рисунок 3.2 – Диаграмма перечислений

В данный модуль вынесены такие перечисления, как *Declension* (склонение), *Gender* (род), *Plurality* (число), *PartOfSpeech* (часть речи), *SuffixType* (тип суффикса) и *SuffixWithEnding* (наличие у суффикса окончания). Представление лингвистических характеристик в виде числовых констант (вместо строк) позволяет значительно повысить скорость выполнения условных проверок в алгоритмах парсинга.

Промежуточные результаты работы поискового алгоритма обрабатываются в модуле структур (*Structures*). Диаграмма структур данных представлена на рисунке 3.3.

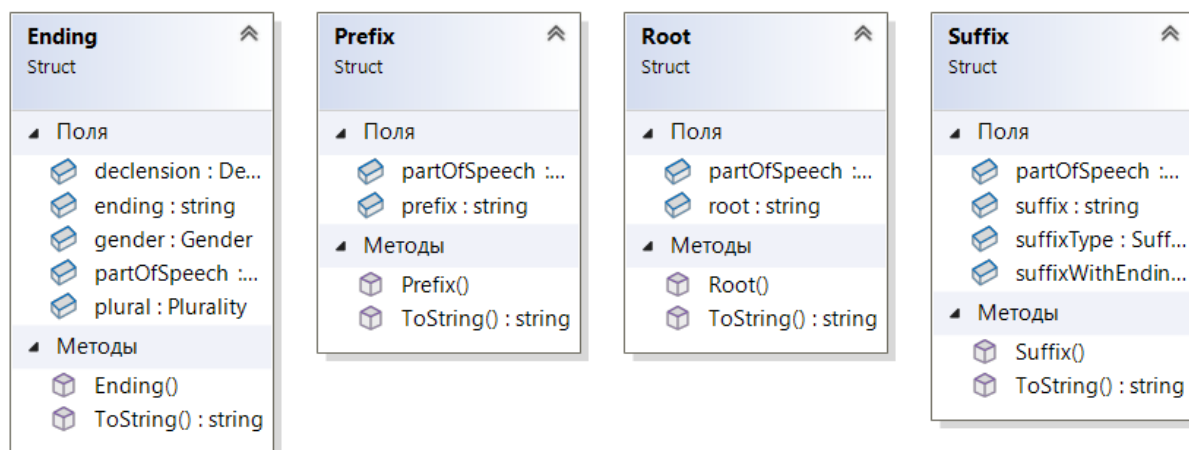


Рисунок 3.3 – Диаграмма структур

В модуле описаны типы значений (*struct*): *Ending*, *Prefix*, *Root*, *Suffix*. Выбор структур вместо классов обусловлен требованиями к производительности. Поскольку структуры размещаются в стеке оперативной памяти, это позволяет избежать избыточной нагрузки на сборщик мусора при генерации объектов-морфем во время алгоритмического разбора текстовых массивов. Каждая структура переопределяет базовый метод *ToString()* для форматированного вывода результатов анализа (это используется на странице с морфемным разбором слова).

Взаимодействие с физической базой данных инкапсулировано в модуле репозитория (*Repositories*). Диаграмма классов слоя доступа к данным представлена на рисунке 3.4.

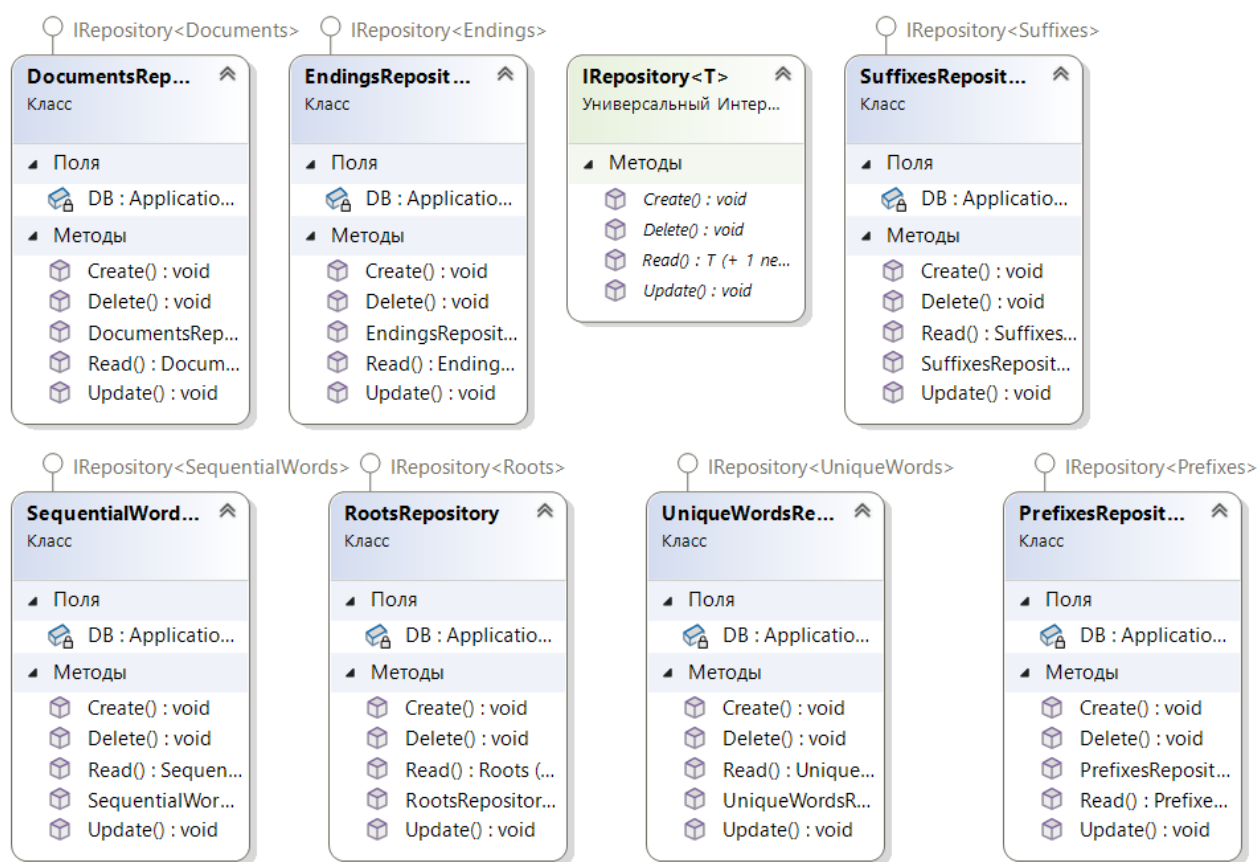


Рисунок 3.4 – Диаграмма классов слоя доступа к данным

Слой реализован с применением обобщенного интерфейса *IRepository<T>*, который определяет стандартный контракт выполнения *CRUD*-операций (создание, чтение, обновление, удаление). Классы *DocumentsRepository*, *RootsRepository*, *SuffixesRepository*, *EndingsRepository*, *SequentialWordsRepository*, *UniqueWordsRepository* и *PrefixesRepository* реализуют данный интерфейс, изолируя *LINQ*-запросы к контексту базы данных (*ApplicationContext*) от остальной части приложения. Такой подход позволяет надежно скрыть сложные детали работы с *Entity Framework* от других модулей системы. Благодаря этому сервисы и бизнес-логика оперируют простыми командами (например, сохранить

объект или найти его по идентификатору), не вдаваясь в особенности формирования *SQL*-запросов. Кроме того, это существенно упрощает поддержку проекта, а именно при любых изменениях в структуре базы данных достаточно будет обновить только соответствующий репозиторий, не затрагивая алгоритм поиска.

Для реализации бизнес-логики приложения и организации безопасного обмена данными между компонентами системы внедрен сервисный слой (*Services*). Диаграмма классов сервисов представлена на рисунке 3.5.

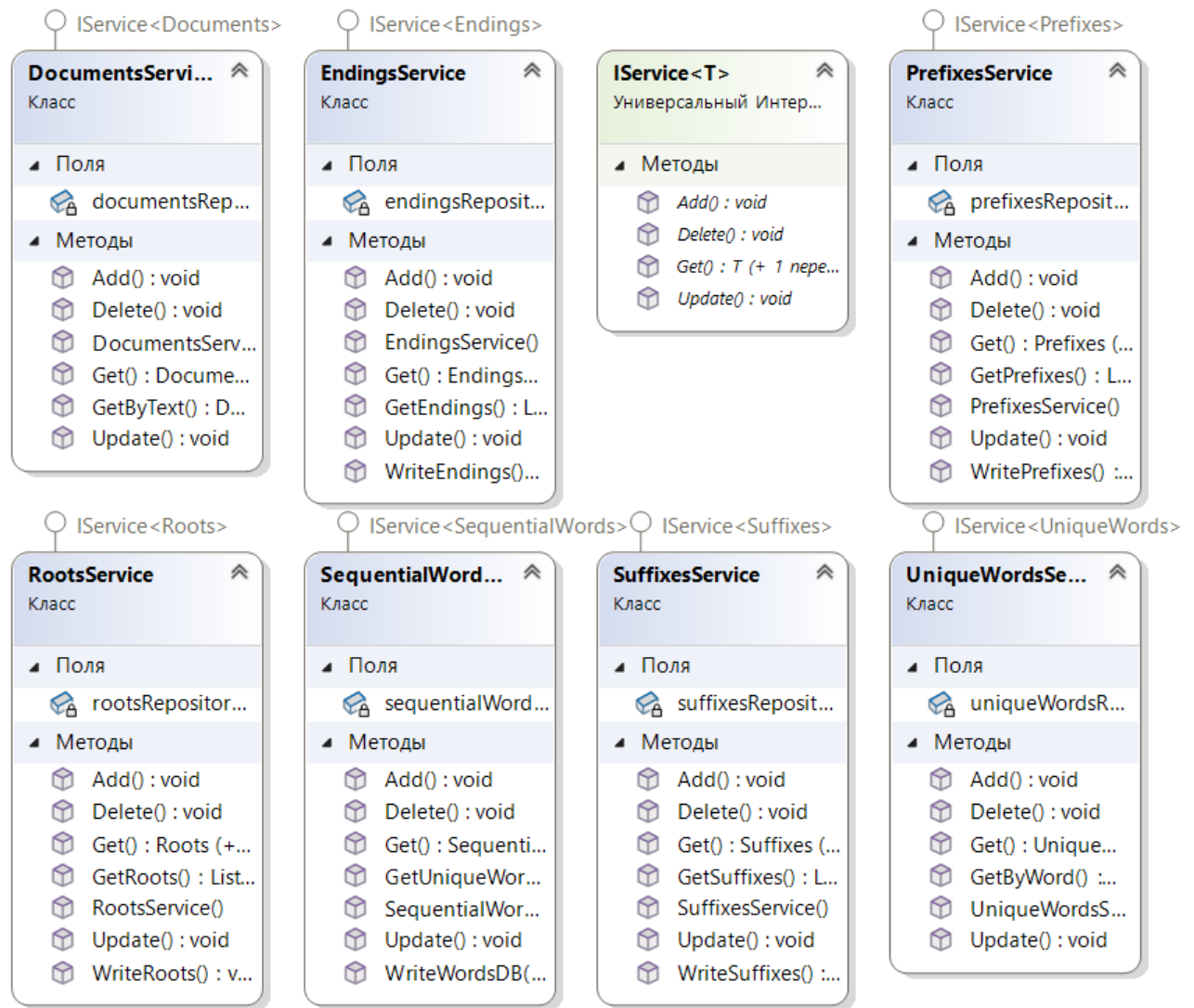


Рисунок 3.5 – Диаграмма классов сервисов

Все сервисы наследуют базовый интерфейс *IService<T>*. Классы данного слоя *DocumentsService*, *UniqueWordsService*, *PrefixesService*, *RootsService*, *EndingsService*, *SuffixesService*, *SequentialWordsService* выступают в роли связующего звена между контроллерами и репозиториями. Внедрение сервисного слоя необходимо для того, чтобы не передавать «сырые» данные напрямую из веб-интерфейса в базу данных. Классы-сервисы выполняют роль

фильтра: они проверяют поступающую информацию на корректность и следят за тем, чтобы в систему не сохранялись дубликаты уже существующих документов или слов.

Центральным интеллектуальным узлом выступает модуль обработки данных (*Classes*), реализующий математическое и лингвистическое ядро поисковика. Диаграмма базовых классов системы представлена на рисунке 3.6.

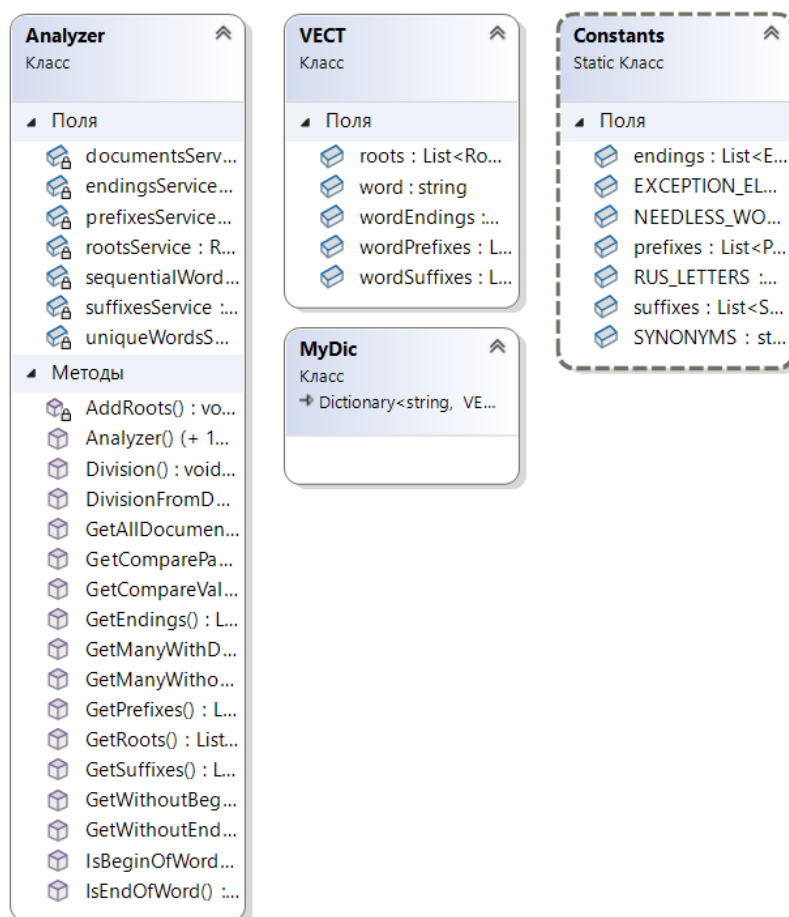


Рисунок 3.6 – Диаграмма базовых классов системы

Модуль включает статический класс *Constants*, хранящий загружаемые в оперативную память словари морфем, массивы «ненужных» слов и списки синонимов. Класс *VECT* представляет собой математический вектор структурных признаков (паспорт слова), а *MyDic* реализует псевдоним типа для словаря векторов *Dictionary<string, VECT>*.

Ключевым классом является *Analyzer*, который содержит основную логику обработки русскоязычных текстов и поиска смысловых совпадений. В структуре данного класса выделяется несколько функциональных групп методов, обеспечивающих полный цикл морфологического анализа и распределения результатов.

Базовые строковые методы, такие как *GetWithoutEnding* и *IsBeginOfWord*, предназначены для проверки наличия заданных подстрок на краях слова. Они

обеспечивают безопасное отсечение символов, исключая возникновение ошибок выхода за границы строки при обработке текста.

Для выделения структурных частей слова применяются методы извлечения морфем: *GetPrefixes*, *GetEndings*, *GetSuffixes* и *GetRoots*. Алгоритм последовательно осуществляет морфологический разбор лексемы.

Связь математических алгоритмов с базой данных обеспечивают методы *Division* и *DivisionFromDB*. При обработке слова система проверяет его наличие в базе данных: если слово уже проиндексировано, ее морфемы извлекаются напрямую из таблиц. При обработке нового слова метод выполняет его полный морфологический разбор, после чего обращается к словарю синонимов и привязывает их корни к исходному слову.

Вычисление метрик релевантности реализуется с помощью методов *GetCompareValue* и *GetCompareParagraph*. На первом этапе алгоритм сопоставляет два слова по их морфемному составу и присваивает им числовые веса, при этом наибольшее значение начисляется за точное совпадение корней. Затем данный принцип применяется для оценки текстовых абзацев: если слова из запроса не просто присутствуют в фрагменте текста, но и образуют последовательность, такой абзац получает максимальный рейтинг и возвращается пользователю в качестве итогового ответа.

Обработка входящих *HTTP*-запросов и координация взаимодействия между пользовательским интерфейсом и бизнес-логикой осуществляются в модуле контроллеров (*Controllers*). Диаграмма контроллеров представлена на рисунке 3.7.

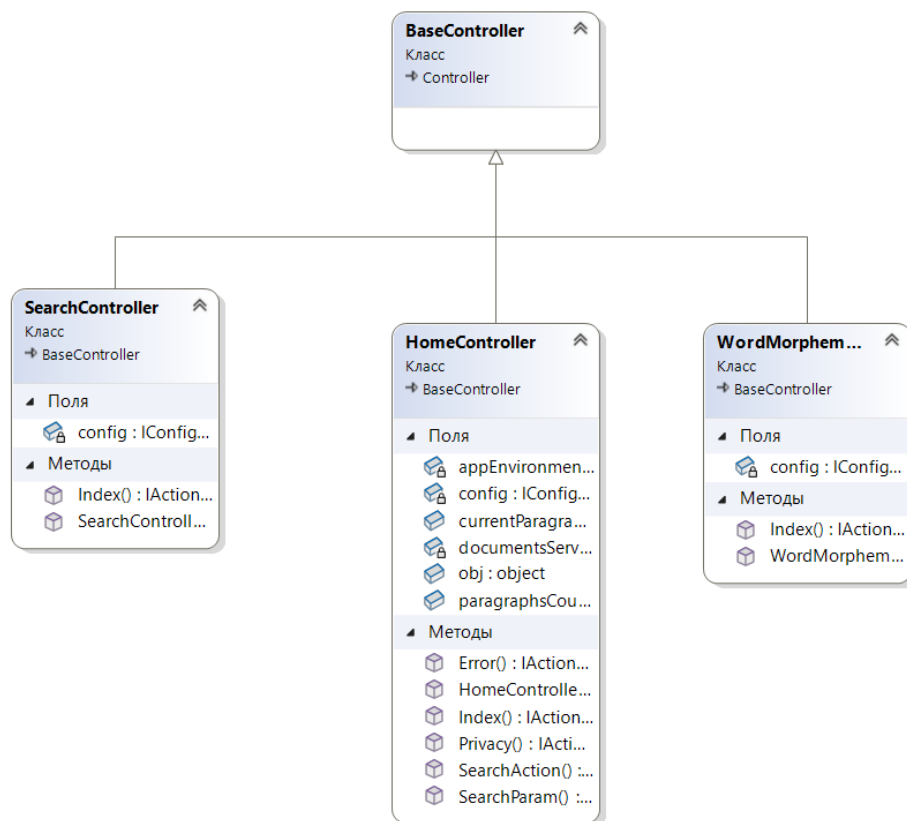


Рисунок 3.7 – Диаграмма контроллеров

Все контроллеры наследуются от базового класса *BaseController*.

Класс *HomeController* отвечает за пополнение базы знаний системы и управляет процессом загрузки новых документов. При получении файла от пользователя данный контроллер выполняет строгую первичную валидацию: проверяет корректность формата (система поддерживает чтение файлов *.txt*, *.docx* и *.pdf*), исключает загрузку пустых документов и проверяет базу данных на наличие дубликатов текстов. После успешного прохождения проверок контроллер извлекает текст из документа, выполняет его нормализацию (приведение к нижнему регистру и замену неоднозначных символов, а также замены ё на е) и передает данные в модуль анализатора. Итогом работы этого контроллера является запуск морфологического разбора текста и сохранение структуры документа в базу данных.

Класс *SearchController* предназначен для обработки поисковых запросов и предоставления результатов пользователю. Получив текстовую строку, контроллер извлекает из базы данных готовые, заранее разобранные векторы слов, что позволяет избежать повторных ресурсоемких вычислений. Затем он вычисляет математическую оценку для каждого абзаца в документах и возвращает фрагмент с наивысшим показателем релевантности. Важной архитектурной особенностью данного модуля является поддержка асинхронного взаимодействия с помощью технологии *Fetch API*. Логика поиска разделена на независимые методы: основной алгоритм выполняет тяжелую вычислительную работу в фоновом режиме, а вспомогательный метод непрерывно отдает данные о количестве обработанных абзацев. Это позволяет обновлять индикатор прогресса (*Progress Bar*) на стороне клиента плавно и без перезагрузки веб-страницы.

Класс *WordMorphemesController* предоставляет вспомогательный функционал, обеспечивающий наглядность и прозрачность работы внутренних алгоритмов системы. Данный модуль позволяет пользователю ввести любое слово на естественном языке и визуально ознакомиться с результатами его декомпозиции. Контроллер перехватывает введенное слово, пропускает его через морфемный анализатор и выводит на экран детальный структурированный список всех идентифицированных морфем: приставок, корней, суффиксов и окончаний.

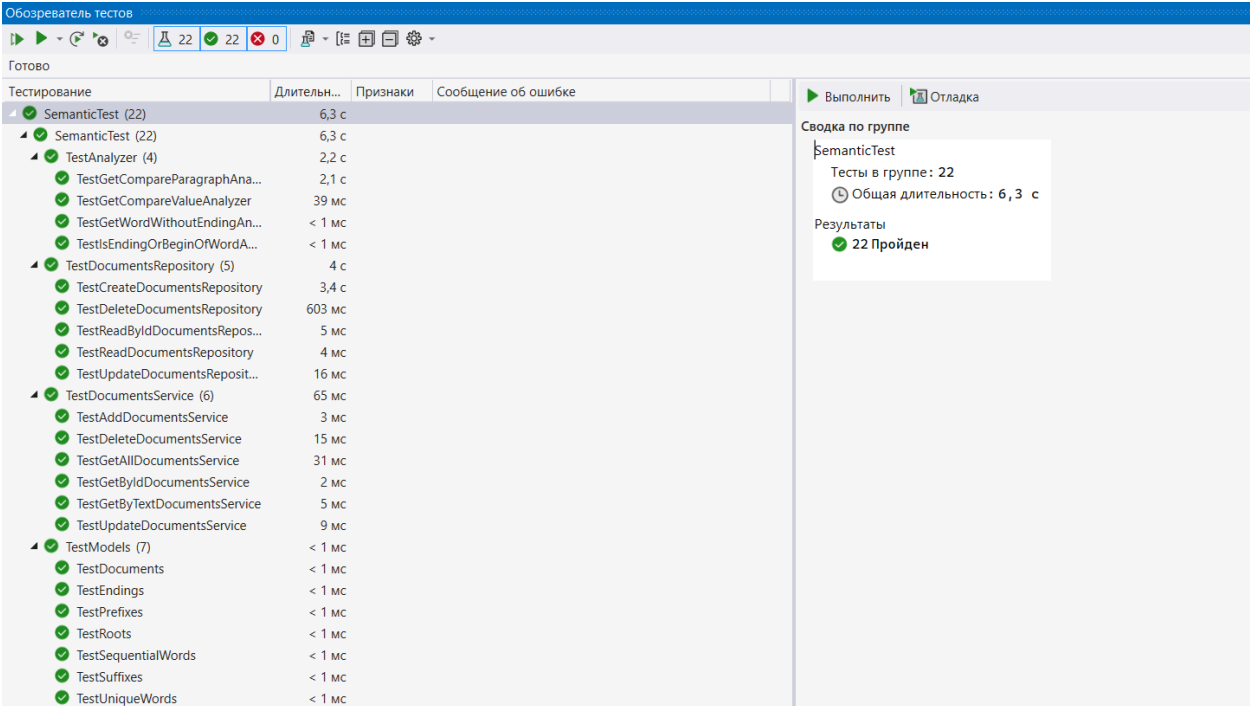
Замыкающим слоем архитектуры является модуль представления пользовательского интерфейса (*Views*). Это та часть приложения, с которой напрямую взаимодействует человек. В данном слое полностью отсутствуют сложные вычислительные алгоритмы или прямые обращения к базам данных – его единственной задачей является удобное и понятное отображение готовой информации в браузере клиента. Для создания веб-страниц используется механизм шаблонизации *Razor* (файлы формата *.cshtml*). Данная технология позволяет органично объединять классическую *HTML*-разметку с данными, полученными от сервера. Например, именно благодаря этому механизму страница динамически отрисовывает текст найденного релевантного абзаца или

подставляет ранее введенный пользователем запрос обратно в поисковую строку.

Графический интерфейс системы спроектирован с применением современных каскадных таблиц стилей (*CSS*). Визуальная часть выстроена таким образом, чтобы быть интуитивно понятной: элементы управления, кнопки и текстовые поля имеют аккуратные отступы и центрирование. Особое внимание уделено обратной связи при пользовательских ошибках (например, при попытке загрузить пустой файл или документ неверного формата). В таких случаях интерфейс не прерывает работу приложения, а выводит на экран соответствующие текстовые предупреждения. Кроме того, визуальный слой поддерживает механизмы асинхронного обновления: благодаря встроенным скриптам страница умеет плавно изменять свое содержимое (например, сдвигать индикатор прогресса при длительном поиске) без полной перезагрузки окна браузера.

3.2 Тестирование программной системы

Для обеспечения корректной работы сложной алгоритмической логики информационной системы необходимо проведение тщательного автоматизированного тестирования. Использование тестовых сценариев гарантирует, что математические формулы расчетов числовых критериев оценки релевантности абзацев, алгоритмы усечения морфем и механизмы кэширования данных в СУБД функционируют без логических ошибок при любых входных данных. Результат успешного выполнения пакета автоматизированных тестов представлен на рисунке 3.8.



Тестирование	Длительн...	Признаки	Сообщение об ошибке
✓ SemanticTest (22)	6,3 с		
✓ SemanticTest (22)	6,3 с		
✓ TestAnalyzer (4)	2,2 с		
✓ TestGetCompareParagraphAna...	2,1 с		
✓ TestGetCompareValueAnalyzer	39 мс		
✓ TestGetWordWithoutEndingAn...	< 1 мс		
✓ TestIsEndingOrBeginOfWordA...	< 1 мс		
✓ TestDocumentsRepository (5)	4 с		
✓ TestCreateDocumentsRepository	3,4 с		
✓ TestDeleteDocumentsRepository	603 мс		
✓ TestReadByldDocumentsRepos...	5 мс		
✓ TestReadDocumentsRepository	4 мс		
✓ TestUpdateDocumentsReposit...	16 мс		
✓ TestDocumentsService (6)	65 мс		
✓ TestAddDocumentsService	3 мс		
✓ TestDeleteDocumentsService	15 мс		
✓ TestGetAllDocumentsService	31 мс		
✓ TestGetByldDocumentsService	2 мс		
✓ TestGetByTextDocumentsService	5 мс		
✓ TestUpdateDocumentsService	9 мс		
✓ TestModels (7)	< 1 мс		
✓ TestDocuments	< 1 мс		
✓ TestEndings	< 1 мс		
✓ TestPrefixes	< 1 мс		
✓ TestRoots	< 1 мс		
✓ TestSequentialWords	< 1 мс		
✓ TestSuffixes	< 1 мс		
✓ TestUniqueWords	< 1 мс		

Выполнить | Отладка

Сводка по группе

SemanticTest

Тесты в группе: 22

Общая длительность: 6,3 с

Результаты

✓ 22 Пройден

Рисунок 3.8 – Результаты выполнения тестов

Тестирование программного комплекса реализовано с применением фреймворка *xUnit*. Структура тестового проекта *SemanticTest* включает в себя статический класс *TestContext*, который реализует паттерн *Singleton* с потокобезопасной блокировкой. Данный механизм отвечает за чтение строки подключения из конфигурационного файла *appsettings.json* и инициализацию единого контекста базы данных для всех тестовых классов. Поскольку тестирование осуществляется с использованием реального файла базы данных, в файле *GlobalUsings.cs* применена директива отключения параллельного выполнения сценариев (*DisableTestParallelization = true*), что предотвращает конфликты блокировки файла СУБД со стороны операционной системы.

Первым логическим уровнем верификации системы выступает тестирование моделей данных, реализованное в классе *TestModels.cs*. Разработаны модульные тесты, подтверждающие целостность объектной структуры классов-сущностей. Выполнение данных сценариев гарантирует корректность инкапсуляции данных в свойствах моделей, выступающих точной проекцией таблиц реляционной базы данных в объектно-ориентированную среду программирования.

Вторым этапом проводится интеграционное тестирование взаимодействия объектно-ориентированного кода с сервером баз данных средствами *Entity Framework Core* (классы *TestDocumentsRepository.cs* и *TestDocumentsService.cs*). Тестовые сценарии построены по классическому паттерну *Arrange-Act-Assert*. Каждый тест генерирует собственный тестовый объект документа, сохраняет его в базу данных и получает автоматически сгенерированный сервером первичный ключ (*Id*). Операции чтения, обновления, удаления выполняются строго по полученному уникальному идентификатору. Особое внимание в данном блоке уделено сервисному слою: отдельный метод *TestGetByTextDocumentsService* проверяет корректность поиска документов по точному текстовому соответствию, что очень важно для надежной работы первичной обработки текста файлов.

Завершающим уровнем является тестирование бизнес-логики морфемного анализатора (класс *TestAnalyzer.cs*). Базовые методы *TestGetWordWithoutEndingAndBeginAnalyzer* и *TestIsEndingOrBeginOfWordAnalyzer* проверяют корректность работы строковых функций усечения концовки слова и определения границ морфем. Далее доказываем строгость математической модели вычисления релевантности слов: алгоритм сопоставляет вектор признаков слова с неодинокоренной лексемой, при этом итоговый результат проверки строго ниже установленного порога в 40 000 единиц (срабатывание *Assert.False*). Ключевым сценарием системы является тест *TestGetCompareParagraphAnalyzer*, эмулирующий полный цикл работы поискового движка. Метод загружает текстовый абзац в базу данных, форматирует его и разбивает на морфемы, после чего сопоставляет абзац с двумя различными поисковыми запросами. Результаты тестирования математически доказывают, что алгоритм корректно вычисляет оценку релевантности: запрос с точным совпадением контекстной последовательности слов получает наивысший балл

(402), в то время как запрос с частичным совпадением получает минимальную оценку (1).

3.3 Апробация разработанной системы

Апробация разработанного программного решения проводилась путем оценки корректности его функционирования при выполнении поиска. Взаимодействие пользователя с системой осуществляется через интуитивно понятный графический веб-интерфейс, спроектированный с использованием технологии стилей *CSS*.

Начальным этапом работы с системой является наполнение базы знаний. На главной странице (*Home*) пользователю или системному администратору предлагается загрузить регламентирующий документ в форматах *.txt*, *.docx* или *.pdf*. При разработке формы загрузки внедрена валидация для предотвращения системных сбоев. Программа последовательно проверяет факт выбора файла, корректность его расширения, наличие текстового содержимого (защита от пустых файлов) и отсутствие дубликатов в базе данных. Пример успешной обработки исключительной ситуации (загрузки пустого файла) с выводом соответствующего информационного сообщения представлен на рисунке В.1.

После успешной обработки документа пользователь может перейти к основному функционалу системы – модулю поиска (*Search*). Интерфейс страницы поиска содержит текстовое поле ввода запроса и кнопку поиска. Алгоритмы контроллера блокируют отправку пустых запросов или строк, состоящих исключительно из пробелов. В случае успешной обработки запроса система выведет на экран наиболее релевантный абзац, состоящий из оригинального текста документа базы знаний. Внешний вид графического интерфейса модуля поиска и пример выдачи релевантного ответа приведены на рисунке В.2. При отсутствии совпадений выводится уведомление о невозможности найти соответствующий контекст.

В качестве вспомогательного инструмента, демонстрирующего внутреннюю логику работы программного комплекса, в систему интегрирован модуль «Морфологический разбор» (*WordMorphemes*). Данный раздел позволяет пользователю ввести любое слово на русском языке и визуальным образом ознакомиться с результатами работы алгоритма отсечения морфем. Приложение выводит списки всех возможных приставок, корней, суффиксов и окончаний с указанием их грамматических характеристик. Визуальное отображение результатов морфемного разбора, введенного пользователем слова, показано на рисунке В.3. Реализация данного функционала повышает удобство использования системы в образовательных целях, а также обеспечивает прозрачность процесса векторизации слов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была успешно достигнута поставленная цель – спроектирована и программно реализована автоматизированная информационная система семантического поиска, связанной с документацией об организации учебного процесса.

В процессе проектирования и разработки программного комплекса были получены следующие практические и теоретические результаты:

- проведен системный анализ предметной области и существующих методов информационного поиска;
- спроектирована многослойная клиент-серверная архитектура на базе фреймворка *ASP.NET Core MVC*;
- разработаны логическая и физическая модели реляционной базы данных, соответствующие требованиям третьей нормальной формы;
- разработан и программно реализован алгоритм морфологического анализа русскоязычных слов;
- успешно решена проблема «семантического разрыва» путем внедрения в алгоритм словарь синонимов;
- разработана математическая модель вычисления релевантности и получения числовых критериев оценки текстовых абзацев;
- разработан универсальный модуль парсинга и индексации файлов, поддерживающий извлечение текстовых данных с сохранением абзацного форматирования из документов форматов *.txt*, *.docx* и *.pdf*;
- проведена комплексная верификация программного обеспечения с использованием фреймворка *xUnit*;
- разработан интуитивно понятный графический веб-интерфейс пользователя, включающий функционал поиска, загрузки документов с защитой от некорректного ввода и визуализации внутреннего процесса морфологического разбора слов.

Список использованных источников

1. Семантический поиск: Свободная энциклопедия. – Электрон. данные. – Режим доступа: https://ru.wikipedia.org/wiki/Семантический_поиск. – Дата доступа: 20.01.2025.
2. Подольский, В. Архитектура высоконагруженных систем / В. Подольский. – М., 2026.
3. Ротман, Д. RAG и генеративный ИИ. Создаем собственные RAG-пайплайны с помощью LlamaIndex, Deep Lake и Pinecone / Д. Ротман. – СПб.: Питер, 2025.
4. Джоши, А. Шаблоны распределенных систем / А. Джоши; пер. с англ. – М.: ДМК Пресс, 2024.
5. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо; пер. с англ. – 2-е изд. – М.: ДМК Пресс, 2023.
6. Прайс, М. Дж. C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals / М. Дж. Прайс. – Birmingham: Packt Publishing, 2024.
7. Любанович, Б. FastAPI: веб-разработка на Python / Б. Любанович. – СПб.: Питер, 2025. – 288 с.
8. Smith, S. Architecting Modern Web Applications with ASP.NET Core and Azure / S. Smith. – Redmond: Microsoft Corporation, 2023. – 119 p.
9. Гринберг, М. SQLAlchemy 2 In Practice: Learn to program relational databases in Python / М. Гринберг. – [S. l.], 2025.
10. Смит, Д. Entity Framework Core in Action / Д. Смит. – 3rd ed. – Shelter Island: Manning Publications, 2025.
11. Applied MongoDB: Data Modeling and Architecture Patterns. – [S. l.], 2025.
12. Башир, С. Learning MySQL: Get a Handle on Your Data / С. Башир. – 2-е изд. – Sebastopol: O'Reilly Media, 2023.
13. Фритчи, Г. SQL Server 2022 Query Performance Tuning / Г. Фритчи. – 6-е изд. – New York: Apress, 2024.
14. Анайя, М. Clean Code in Python / М. Анайя. – 3-е изд. – Birmingham: Packt Publishing, 2025.
15. Test-Driven Development in .NET with xUnit. – Birmingham: Packt Publishing, 2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы

Analyzer.cs:

```
using DocumentFormat.OpenXml.Drawing.Diagrams;
using Microsoft.AspNetCore.Routing;
using Microsoft.EntityFrameworkCore;
using SemanticSearch.Enums;
using SemanticSearch.Models;
using SemanticSearch.Services;
using SemanticSearch.Structures;
using System.Collections.Generic;
using System.Runtime.InteropServices;

namespace SemanticSearch.Classes
{
    public class Analyzer
    {
        private readonly DocumentsService documentsService;
        private readonly UniqueWordsService uniqueWordsService;
        private readonly PrefixesService prefixesService;
        private readonly SequentialWordsService sequentialWordsService;
        private readonly RootsService rootsService;
        private readonly EndingsService endingsService;
        private readonly SuffixesService suffixesService;

        public Analyzer(ApplicationContext DB)
        {
            documentsService = new DocumentsService(DB);
            uniqueWordsService = new UniqueWordsService(DB);
            prefixesService = new PrefixesService(DB);
            sequentialWordsService = new SequentialWordsService(DB);
            rootsService = new RootsService(DB);
            endingsService = new EndingsService(DB);
            suffixesService = new SuffixesService(DB);
        }

        public Analyzer(IConfiguration config)
        {
            var DB = new ApplicationContext(config.GetConnectionString("ConnectionString"));
            documentsService = new DocumentsService(DB);
            uniqueWordsService = new UniqueWordsService(DB);
            prefixesService = new PrefixesService(DB);
            sequentialWordsService = new SequentialWordsService(DB);
            rootsService = new RootsService(DB);
            endingsService = new EndingsService(DB);
            suffixesService = new SuffixesService(DB);
        }

        // Проверяем концовку слова
        public bool IsEndOfWord(string word, string ending)
            => word.EndsWith(ending) && (word.Length > ending.Length);

        // получаем начало слова
        public bool IsBeginOfWord(string word, string begin)
```



```

=> word.StartsWith(begin) && (word.Length > begin.Length); // приставка должна быть меньше, чем
само слово

// Возвращаем часть слова без указанной концовки
public string GetWithoutEnding(string word, string ending)
=> word.Substring(0, word.Length - ending.Length);

// Возвращаем часть слова без указанного начала
public string GetWithoutBegin(string word, string begin)
=> word.Substring(begin.Length, word.Length - begin.Length);

// Ищем все возможные окончания слова
public List<Ending> GetEndings(string word)
=> Constants.endings.FindAll(e => IsEndOfWord(word, e.ending))
.ToList();

// Ищем все возможные приставки слова
public List<Prefix> GetPrefixes(string word)
=> Constants.prefixes.FindAll(p => IsBeginOfWord(word, p.prefix))
.ToList();

// Ищем все возможные суффиксы слов.
// wordEndings - окончания заданного слова word.
public List<Suffix> GetSuffixes(string word, List<Ending> wordEndings)
{
    var result = new List<Suffix>();

    // Составляем комбинации суффиксов с окончаниями
    var suffixVariants = new List<Suffix>();
    foreach (var suffix1 in Constants.suffixes) // Перебираем все суффиксы
    {
        if (suffix1.suffixWithEnding == SuffixWithEnding.WITH_ENDING)
        {
            suffixVariants.Add(suffix1);
            foreach (var suffix2 in Constants.suffixes) // Перебираем все суффиксы
            {
                if ((suffix2.suffixWithEnding == SuffixWithEnding.WITH_ENDING) && (suffix1.partOfSpeech ==
suffix2.partOfSpeech))
                {
                    Suffix doubleSuffix;
                    suffixVariants.Add(doubleSuffix = new Suffix(
                        suffix1.suffix + suffix2.suffix,
                        SuffixType.DIMINUTIVE,
                        SuffixWithEnding.WITH_ENDING,
                        suffix1.partOfSpeech));

                    foreach (var suffix3 in Constants.suffixes) // Перебираем все суффиксы
                    {
                        if ((suffix3.suffixWithEnding == SuffixWithEnding.WITH_ENDING) &&
(doubleSuffix.partOfSpeech == suffix3.partOfSpeech))
                        {
                            suffixVariants.Add(new Suffix(
                                doubleSuffix.suffix + suffix3.suffix,
                                SuffixType.DIMINUTIVE,
                                SuffixWithEnding.WITH_ENDING,
                                doubleSuffix.partOfSpeech));
                        }
                    }
                }
            }
        }
    }
}

```

```

// Для суффиксов с возможными окончаниями
foreach (var suffix in suffixVariants) // Перебираем все суффиксы.
{
    foreach (var ending in wordEndings)
    {
        if ((suffix.partOfSpeech == ending.partOfSpeech) && (word.Length > ending.ending.Length))
        {
            var wordWithoutEnding = GetWithoutEnding(word, ending.ending);

            if (IsEndOfWord(wordWithoutEnding, suffix.suffix))
            {
                result.Add(suffix);
            }
        }
    }
}

// Для суффиксов без дальнейших окончаний
foreach (var suffix in Constants.suffixes) // Перебираем все суффиксы
{
    if (suffix.suffixWithEnding == SuffixWithEnding.WITHOUT_ENDING)
    {
        if (IsEndOfWord(word, suffix.suffix))
        {
            result.Add(suffix);
        }
    }
}
return result;
}

// Добавляем корни, удаляя суффиксы, полагая что они все являются концовкой части переданного слова.
private void AddRoots(string partOfWord, PartOfSpeech partOfSpeech,
    List<Suffix> wordSuffixes, List<Root> roots,
    SuffixWithEnding suffixWithEnding)
{
    foreach (var suffix in wordSuffixes) // по всем суффиксам
    {
        if ((suffix.partOfSpeech == partOfSpeech) &&
            (suffix.suffixWithEnding == suffixWithEnding) &&
            (partOfWord.Length > suffix.suffix.Length))
        {
            var wordWithoutSuffix = GetWithoutEnding(partOfWord, suffix.suffix);
            roots.Add(new Root(wordWithoutSuffix, suffix.partOfSpeech));
        }
    }
}

private void AddRoots(string partOfWord,
    PartOfSpeech partOfSpeech, List<Ending> wordEndings,
    List<Suffix> wordSuffixes, List<Root> roots)
{
    foreach (var ending in wordEndings) // по всем окончаниям
    {
        if (partOfWord.Length > ending.ending.Length && ending.partOfSpeech == partOfSpeech)
        {
            var wordWithoutEnding = GetWithoutEnding(partOfWord, ending.ending);

            if (wordSuffixes.Any())

```

```

        AddRoots(wordWithoutEnding, ending.partOfSpeech, wordSuffixes, roots,
        SuffixWithEnding.WITH_ENDING);

        roots.Add(new Root(wordWithoutEnding, ending.partOfSpeech));
    }
}

public List<Root> GetRoots(string word,
    List<Prefix> wordPrefixes, List<Suffix> wordSuffixes,
    List<Ending> wordEndings)
{
    List<Root> result = new List<Root>();

    // Если есть приставки
    if (wordPrefixes.Any())
    {
        foreach (var prefix in wordPrefixes) // по всем приставкам
        {
            if (word.Length > prefix.prefix.Length)
            {
                var wordWithoutPrefix = GetWithoutBegin(word, prefix.prefix);
                if (wordEndings.Any())
                {
                    AddRoots(wordWithoutPrefix, prefix.partOfSpeech, wordEndings, wordSuffixes, result);
                }
                else // Нет окончаний
                {
                    if (wordSuffixes.Any())
                        AddRoots(wordWithoutPrefix, prefix.partOfSpeech, wordSuffixes, result,
                        SuffixWithEnding.WITHOUT_ENDING);

                    // Нет суффикса и окончания, либо он ошибочный
                    result.Add(new Root(wordWithoutPrefix, prefix.partOfSpeech));
                }
            }
        }
    }

    // Если нет приставок
    // Или в любом случае добавляем корни, полагая, что приставки найдены ошибочно.
    // То есть приставки не рассматриваем

    if (wordEndings.Any()) // Есть окончания
    {
        AddRoots(word, PartOfSpeech.NOUN, wordEndings, wordSuffixes, result);
        AddRoots(word, PartOfSpeech.VERB, wordEndings, wordSuffixes, result);
        AddRoots(word, PartOfSpeech.OTHER, wordEndings, wordSuffixes, result);
    }

    if (wordSuffixes.Any()) // Есть суффиксы
    {
        AddRoots(word, PartOfSpeech.NOUN, wordSuffixes, result, SuffixWithEnding.WITHOUT_ENDING);
        AddRoots(word, PartOfSpeech.VERB, wordSuffixes, result, SuffixWithEnding.WITHOUT_ENDING);
        AddRoots(word, PartOfSpeech.OTHER, wordSuffixes, result, SuffixWithEnding.WITHOUT_ENDING);
    }

    // И как единый корень
    result.Add(new Root(word, PartOfSpeech.NOUN));
    result.Add(new Root(word, PartOfSpeech.VERB));
    result.Add(new Root(word, PartOfSpeech.OTHER));
}

```

```

        // Сразу удаляем дубликаты
        return result.Distinct().ToList();
    }

    public void Division(string word,
        out List<Prefix> wordPrefixes,
        out List<Root> wordRoots,
        out List<Suffix> wordSuffixes,
        out List<Ending> wordEndings)
    {
        word = word.ToLower();
        word = word.Replace("ё", "е");

        wordEndings = GetEndings(word);
        wordSuffixes = GetSuffixes(word, wordEndings);
        wordPrefixes = GetPrefixes(word);
        wordRoots = GetRoots(word, wordPrefixes, wordSuffixes, wordEndings);
    }

    // Разбор оригинального текста файлы базы с занесением морфем в БД.
    // Выполняется при загрузке файла.
    // dic - словарь корней слов и их корней-синонимов слов.

    public MyDic Division(string[] words, Dictionary<string, List<string>> dic)
    {
        var result = new MyDic();

        foreach (var word in words)
        {
            result.Add(word, Division(word, dic));
        }

        return result;
    }

    // Выполняется при разборе оригинального текста из файла, для слов которого морфемы уже занесены в
    БД.
    public MyDic DivisionFromDB(string[] words)
    {
        var result = new MyDic();

        foreach (var word in words)
            result.Add(word, DivisionFromDB(word));

        return result;
    }

    public int GetCompareValue(string word1, VECT v2) // v2 - по второму слову.
    {
        Division(word1, out List<Prefix> wordPrefixes1, out List<Root> roots1, out List<Suffix> wordSuffixes1,
            out List<Ending> wordEndings1);

        var values = new int[4] { 0, 0, 0, 0 };

        foreach (var root1 in roots1)

```

```

    {
        foreach (var root2 in v2.roots)
        {
            if (root1.root == root2.root /*&& root1.partOfSpeech == root2.partOfSpeech*/)
            {
                values[0] = 1;
            }
        }
    }

    foreach (var prefix1 in wordPrefixes1)
    {
        foreach (var prefix2 in v2.wordPrefixes)
        {
            if ((prefix1.prefix == prefix2.prefix) &&
                prefix1.partOfSpeech == prefix2.partOfSpeech)
            {
                values[1] = 1;
            }
        }
    }

    foreach (var suffix1 in wordSuffixes1)
    {
        foreach (var suffix2 in v2.wordSuffixes)
        {
            if ((suffix1.suffix == suffix2.suffix) &&
                suffix1.partOfSpeech == suffix2.partOfSpeech)
            {
                values[2] = 1;
            }
        }
    }

    foreach (var ending1 in wordEndings1)
    {
        foreach (var ending2 in v2.wordEndings)
        {
            if ((ending1.ending == ending2.ending) &&
                (ending1.partOfSpeech == ending2.partOfSpeech) &&
                ending1.plural == ending2.plural)
            {
                values[3] = 1;
            }
        }
    }

    var result = 0;
    for (int i = 0; i < 4; i++)
    {
        result += values[i] * (4 - i) * (int)Math.Pow(10, 4 - i);
    }

    return result;
}

// Получаем разбиение из базы или HOBOE и заносим в базу
public VECT Division(string word, Dictionary<string, List<string>> dic)
{
    word = word.ToLower().Replace("ё", "е");

    VECT v = new VECT();

```

```

// проверяем, есть ли слово в таблице "Слова не дубликаты"
var uniqueWords = uniqueWordsService.GetByWord(word);

if (uniqueWords != null)
{
    v.wordPrefixes = prefixesService.GetPrefixes(uniqueWords.Id);
    v.roots = rootsService.GetRoots(uniqueWords.Id);
    v.wordEndings = endingsService.GetEndings(uniqueWords.Id);
    v.wordSuffixes = suffixesService.GetSuffixes(uniqueWords.Id);
}
else
{
    Division(word, out v.wordPrefixes, out v.roots, out v.wordSuffixes, out v.wordEndings);

    if (dic != null) // проверяем массив на существование
    {
        var newRoots = new List<string>();
        foreach (var root in v.roots)
        {
            var wordRoot = root.root;
            if (dic.ContainsKey(wordRoot))
            {
                newRoots.AddRange(dic[wordRoot]);
            }
        }
        foreach (var newRoot in newRoots)
        {
            v.roots.Add(new Root
            {
                root = newRoot,
                partOfSpeech = PartOfSpeech.NONE /* Без разницы часть речи. */
            });
        }
    }

    // записываем слово
    uniqueWordsService.Add(new UniqueWords()
    {
        OriginalWord = word,
    });

    // получаем его ID
    uniqueWords = uniqueWordsService.GetByWord(word);

    // Здесь уже никак не может быть null, иначе уже ошибка БД
    if (uniqueWords == null)
        throw new Exception("Ошибка БД");

    // записываем морфемы в БД
    prefixesService.WritePrefixes(uniqueWords.Id, v.wordPrefixes);
    rootsService.WriteRoots(uniqueWords.Id, v.roots);
    endingsService.WriteEndings(uniqueWords.Id, v.wordEndings);
    suffixesService.WriteSuffixes(uniqueWords.Id, v.wordSuffixes);
}

return v;
}

```

```

// Получаем разбиение из базы или HOBOE и заносим в базу
public VECT DivisionFromDB(string word)
{
    word = word.ToLower().Replace("ё", "е");

    VECT v = new VECT();

    var uniqueWords = uniqueWordsService.GetByWord(word);
    v.wordPrefixes = prefixesService.GetPrefixes(uniqueWords.Id);
    v.roots = rootsService.GetRoots(uniqueWords.Id);
    v.wordEndings = endingsService.GetEndings(uniqueWords.Id);
    v.wordSuffixes = suffixesService.GetSuffixes(uniqueWords.Id);
    return v;
}

public List<Documents> GetAllDocuments()
{
    return documentsService.Get();
}

// Получаем форматированные слова без дублей по тексту
public string[] GetManyWithDB(string originalText)
{
    // Ищем уже занесённый документ с текстом в базе данных.
    var document = documentsService.GetByText(originalText);

    // Текст в базе данных
    if (document != null)
    {
        // получить выборку слов из базы данных, связанных с document по documents.Id
        return sequentialWordsService
            .GetUniqueWordsByDocumentsId(document.Id);
    }

    else
    {
        var textInLower = originalText.ToLower().Replace("ё", "е");
        var wordsI = textInLower.Split(Constants.EXCEPTION_ELEMENTS); // Слов с пробелами быть не
        может!
        var all = wordsI
            .Where(w => w.Length > 2)
            .Where(w => !Constants.NEEDLESS_WORDS.Contains(w))
            .Distinct();

        // записываем ОРИГИНАЛЬНЫЙ текст в документ
        documentsService.Add(new Documents()
        {
            OriginalText = originalText,
            DateAdded = DateTime.Now,
            Title = "Документ на " + DateTime.Now.ToString("dd.MM.yyyy")
        });

        // получаем ID этого документа
        document = documentsService.GetByText(originalText);

        if (document == null)
            throw new("Ошибка БД");

        //записываем все слова в БД
        sequentialWordsService.WriteWordsDB(document.Id, all);
    }
}

```

```

        return all.ToArray();
    }
}

// text - оригинал.
public string[] GetManyWithoutDB(string text)
{
    text = text.ToLower().Replace("ё", "е");

    var words1 = text.Split(Constants.EXCEPTION_ELEMENTS);

    // Уникальные не делаем, сохраняем последовательность и дубли
    var all = words1
        .Select(w => w.Trim())
        .Where(w => w.Length > 2)
        .Where(w => !Constants.NEEDLESS_WORDS.Contains(w));

    return all.ToArray();
}

// Главный начальный метод получения числовой оценки критерия абзаца text2
// text1 - оригинал запроса
// text2 - оригинал абзаца из текста - возможного ответа.
public int GetCompareParagraph(string textQuery, string textDocument, MyDic dic)
{
    var wordsQuery = GetManyWithoutDB(textQuery); // Дубли не убираем
    var wordsDocument = GetManyWithoutDB(textDocument);

    // Список найденных (встречающихся) слов.
    var foundWords = new List<string>();

    // 1 * x - кол-во слов из запроса в тексте
    var vector1 = 0;

    // Таблица взаимооценок слов.
    var table = new Dictionary<string, Dictionary<string, int>>();

    foreach (var wordQuery in wordsQuery)
    {
        int r_max = 0;
        foreach (var wordDocument in wordsDocument)
        {
            int r_cur = GetCompareValue(wordQuery, dic[wordDocument]);

            // Оценки слов в запросе по отношению к словам в тексте храним в словаре.
            if (!table.ContainsKey(wordQuery))
            {
                table.Add(wordQuery, new Dictionary<string, int>());
            }
            var tableWordDic = table[wordQuery];
            if (!tableWordDic.ContainsKey(wordDocument))
            {
                tableWordDic.Add(wordDocument, r_cur);
            }

            r_max = r_cur > r_max ? r_cur : r_max;
        }
        if (r_max >= 40000)
        {
            vector1++;
            // Добавляем в список найденных слов.
            foundWords.Add(wordQuery);
        }
    }
}

```



```

    }
}

// Подменяем запрос на найденные слова для
// дальнейшего анализа последовательностей!!!!
wordsQuery = foundWords.ToArray();

var vector2 = 0;
if (vector1 > 0) // Если слова есть (хотя бы одно)
{
    // Количество слов из запроса
    for (var sequenceLength = 2; sequenceLength <= wordsQuery.Count(); sequenceLength++)
    {
        var sequenceStart = 0;

        // такую же последовательность слов в тексте
        for (var indexWords2 = 0; indexWords2 <= wordsDocument.Length - sequenceLength; indexWords2++)
        {
            // проверяем совпадение последовательности
            var indexSequence = 0;
            while (indexSequence < sequenceLength) {

                var wordQ = wordsQuery[sequenceStart + indexSequence];
                var wordD = wordsDocument[indexWords2 + indexSequence];

                // Наверняка есть взаимооценка двух слов!
                if (table[wordQ][wordD] < 40000) break;

                indexSequence++;
            }
            if (indexSequence == sequenceLength)
            {
                vector2 += (int)Math.Pow(2, indexSequence);
            }
        }
    }
}

vector2 *= 100; // нахождение последовательностей делаем более релевантнее

return vector1 + vector2;
/*
1 * x - кол-во слов из запроса в тексте
100 * x - кол-во последовательностей слов запроса в тексте от 2 до кол-ва слов
*/
}
}
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Описание таблиц базы данных

Таблица Б.1 – Структура таблицы «*Documents*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор документа	<i>INT</i> <i>IDENTITY(1,1)</i>	<i>PK, NOT NULL</i>
<i>Title</i>	Название загруженного документа	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>
<i>DateAdded</i>	Время добавления	<i>DATETIME2</i>	<i>NOT NULL</i>
<i>OriginalText</i>	Полный оригинальный текст документа	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>

Таблица Б.2 – Структура таблицы «*SequentialWords*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор записи	<i>INT</i> <i>IDENTITY(1,1)</i>	<i>PK, NOT NULL</i>
<i>Word</i>	Слова из текста в исходном виде	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>
<i>DocumentsId</i>	Ссылка на документ	<i>INT</i>	<i>FK, NOT NULL</i>

Таблица Б.3 – Структура таблицы «*UniqueWords*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор слова	<i>INT</i> <i>IDENTITY(1,1)</i>	<i>PK, NOT NULL</i>
<i>OriginalWord</i>	Оригинал слова	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>

Таблица Б.4 – Структура таблицы «*Roots*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор корня	<i>INT</i> <i>IDENTITY(1,1)</i>	<i>PK, NOT NULL</i>
<i>Root</i>	Корень	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>
<i>PartOfSpeechId</i>	Идентификатор части речи	<i>INT</i>	<i>NOT NULL</i>
<i>UniqueWordsId</i>	Ссылка на слово	<i>INT</i>	<i>FK, NOT NULL</i>

Таблица Б.5 – Структура таблицы «*Prefixes*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор приставки	<i>INT</i> <i>IDENTITY(1,1)</i>	<i>PK, NOT NULL</i>
<i>Prefix</i>	Приставка	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>
<i>PartOfSpeechId</i>	Идентификатор части речи	<i>INT</i>	<i>NOT NULL</i>
<i>UniqueWordsId</i>	Ссылка на слово	<i>INT</i>	<i>FK, NOT NULL</i>

Таблица Б.6 – Структура таблицы «*Suffixes*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор суффикса	<i>INT</i>	<i>PK, NOT NULL</i>
<i>Suffix</i>	Суффикс	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>
<i>SuffixTypeId</i>	Тип суффикса	<i>INT</i>	<i>NOT NULL</i>
<i>PartOfSpeechId</i>	Идентификатор части речи	<i>INT</i>	<i>NOT NULL</i>
<i>UniqueWordsId</i>	Ссылка на слово	<i>INT</i>	<i>FK, NOT NULL</i>

Таблица Б.7 – Структура таблицы «*Endings*»

Имя поля	Описание	Тип данных	Ограничения
<i>Id</i>	Уникальный идентификатор окончания	<i>INT</i> <i>IDENTITY(1,1)</i>	<i>PK, NOT NULL</i>
<i>Ending</i>	Окончание	<i>NVARCHAR(MAX)</i>	<i>NOT NULL</i>
<i>GenderId</i>	Идентификатор грамматического рода	<i>INT</i>	<i>NOT NULL</i>
<i>PluraliryId</i>	Идентификатор грамматического числа	<i>INT</i>	<i>NOT NULL</i>
<i>DeclensionId</i>	Идентификатор типа склонения	<i>INT</i>	<i>NOT NULL</i>
<i>PartOfSpeechId</i>	Идентификатор части речи	<i>INT</i>	<i>NOT NULL</i>
<i>UniqueWordsId</i>	Ссылка на слово	<i>INT</i>	<i>FK, NOT NULL</i>

ПРИЛОЖЕНИЕ В

(обязательное)

Внешний вид графического интерфейса программы

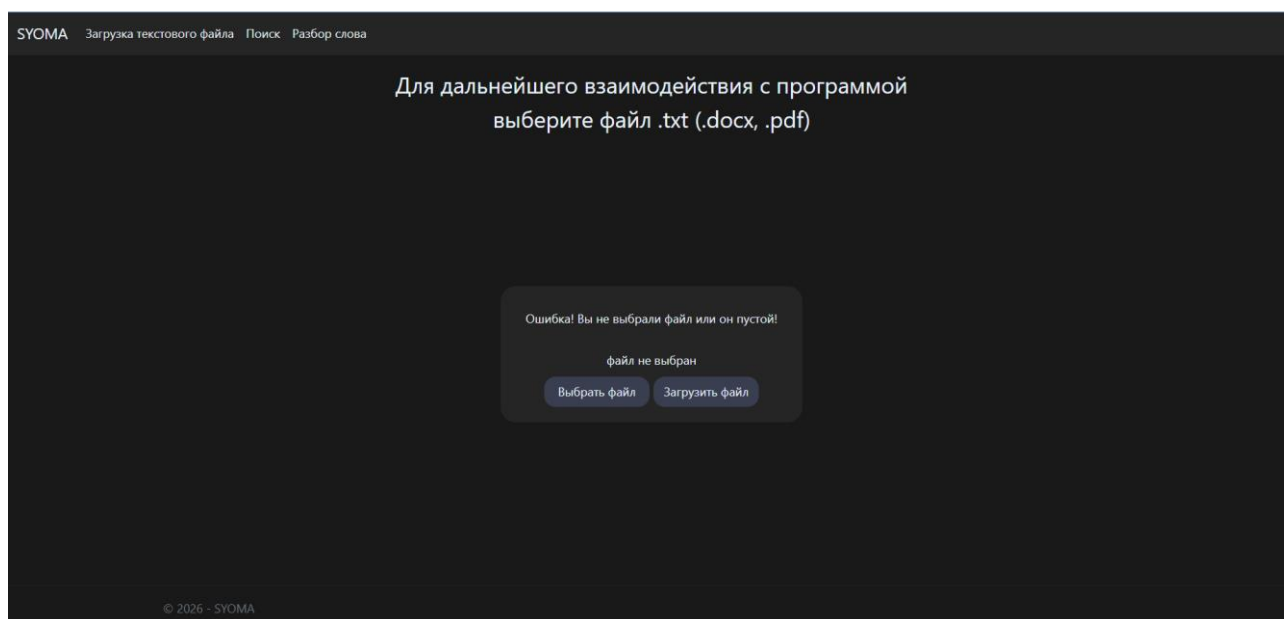


Рисунок В.1 – Успешная обработка загрузки пустого файла

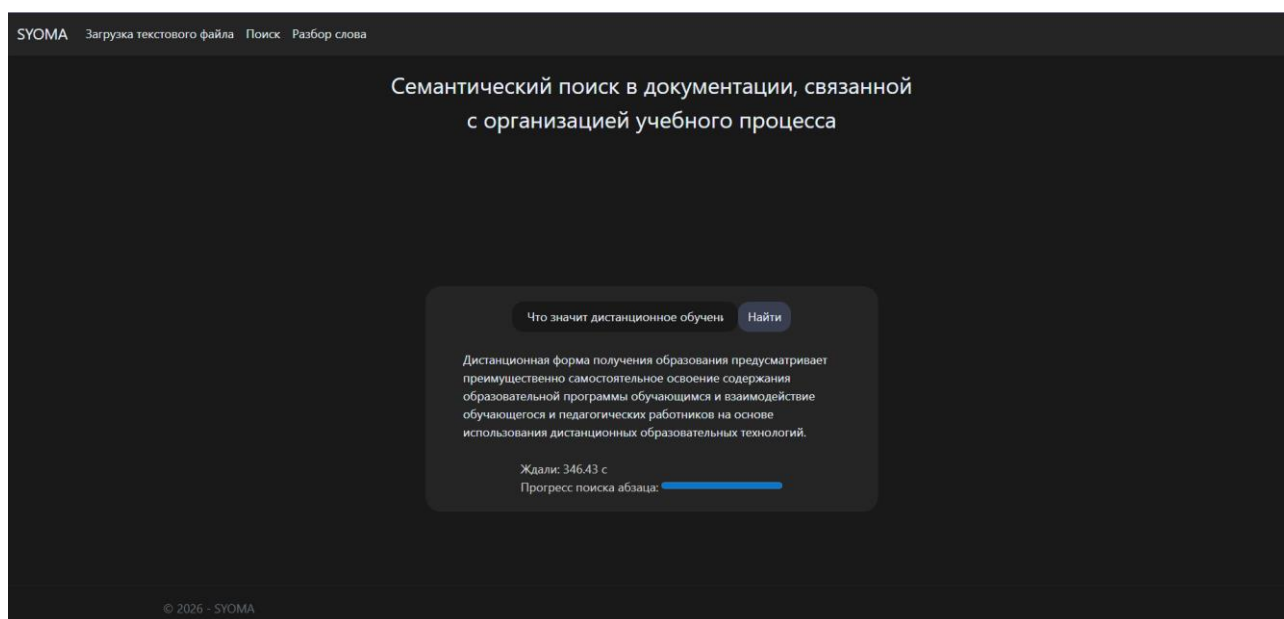


Рисунок В.2 – Результат поиска

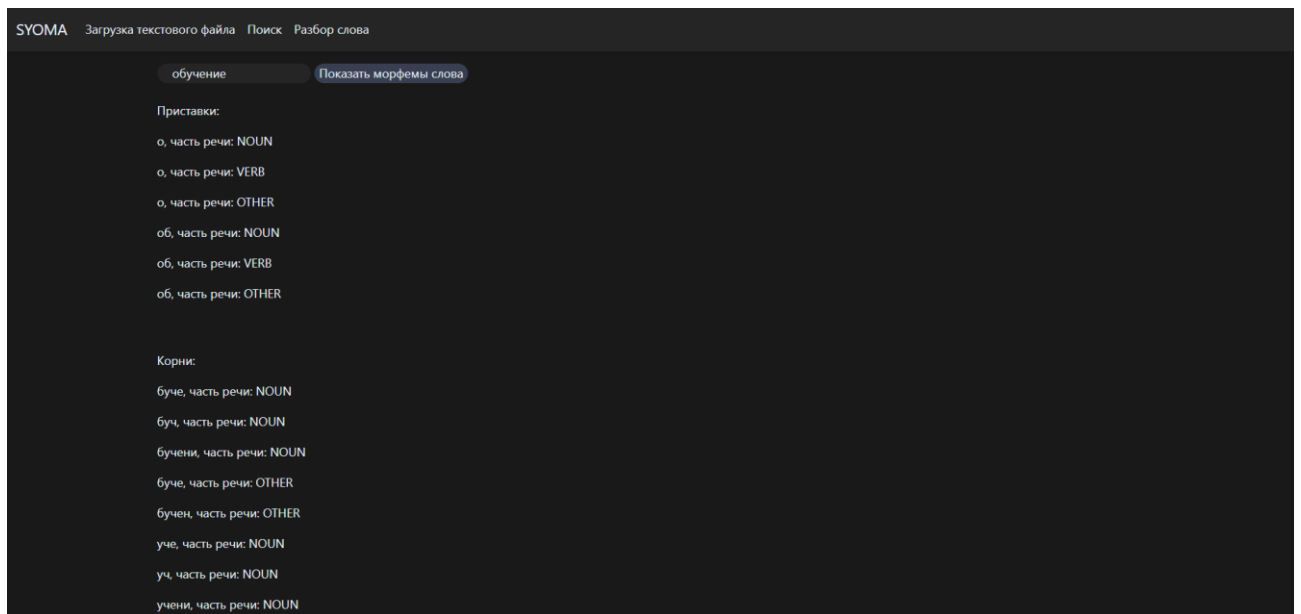


Рисунок В.3 – Результат морфемного разбора слова